Gabriel Kenji de Almeida

Implementação do compilador C-.

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Outubro de 2020

Resumo

O presente relatório tem por finalidade descrever como ocorreu o desenvolvimento e implementação de um sistema computacional munido de processador, com um conjunto de instruções capaz de executar uma variedade de operações aritméticas, lógicas e de gerenciamento de memória, e de um compilador, capaz de interpretar a linguagem de programação simplificada C-, gerando código de máquina adaptado as instruções que o processador desenvolvido pode vir a executar. O processador foi simulado no FPGA (Field Programmable Gate Array), um chip reprogramável a nível de portas lógicas, por meio do uso do Verilog, uma linguagem de descrição de hardware. Todos os conceitos e métodos utilizados serão devidamente explicados posteriormente. O compilador foi implementado utilizando-se C, em conjunto com as ferramentas Bison e Flex, responsáveis pela análise léxica e sintática, respectivamente. Esses conceitos serão esclarecidos em um outro momento. A elaboração do projeto se deu, primeiramente, pelo projeto e implementação do processador, na disciplina de Laboratório de Sistemas Computacionais: Arquitetura e Organização de computadores, realizada durante um semestre completo. Ademais, a implementação do módulo de análise do compilador foi desenvolvida na disciplina de Compiladores, e o módulo de síntese na disciplina de Laboratório de Sistemas Computacionais: Compiladores. O presente relatório possui o intuito de apresentar mais detalhes das etapas de implementação do compilador. O processador será abordado de modo mais superficial.

Palavras-chaves: MIPS. Verilog. Processador. Quartus Prime. Compilador. C-.

Lista de ilustrações

Figura 1 – Caminho de Dados, elaborado com a ferramenta <draw.io> 11
Figura 2 – Diagrama em bloco do módulo de análise, elaborado com a ferramenta <draw.io>. Fonte: O Autor 19
Figura 3 – Diagrama de atividades do módulo de análise, elaborado com a ferramenta <draw.io>. Fonte: O Autor 20
Figura 4 – Diagrama em bloco do módulo de síntese, elaborado com a ferramenta <draw.io>. Fonte: O Autor 24
Figura 5 – Diagrama de atividades do módulo de síntese, elaborado com a ferramenta <draw.io>. Fonte: O Autor 25

Lista de tabelas

Tabela 1 – Operações realizadas pelo código intermediário 27

Sumário

1 INTRODUÇÃO . . . 9

2 O PROCESSADOR . . . 11
2.1 Módulos do processador . . . 12
2.1.1 Unidade de controle . . . 12
2.1.2 Program Counter . . . 12
2.1.3 Memória . . . 12
2.1.4 Banco de Registradores . . . 12
2.1.5 Unidade lógica e aritmética . . . 12
2.2 Interação entre os elementos do caminho de dados . . . 12
2.3 Conjunto de instruções escolhido e suas particularidades . . . 14
2.4 Organização da memória . . . 16

3 COMPILADOR: FASE DE ANÁLISE . . . 17
3.1 Análise léxica . . . 17
3.2 Análise sintática . . . 17
3.3 Análise semântica . . . 18
3.4 Modelagem . . . 18

4 FASE DE SÍNTESE . . . 23
4.1 Modelagem . . . 23
4.2 Geração de código intermediário . . . 26
4.3 Geração de código assembly . . . 27
4.4 Geração de código binário . . . 28
4.5 Gerenciamento de memória . . . 28

5 EXEMPLOS . . . 31
5.1 Exemplo 1: GCD . . . 31
5.1.1 Código em C- . . . 31
5.1.2 Código intermediário . . . 31
5.1.3 Código assembly . . . 32
5.1.4 Código binário . . . 34
5.2 Exemplo 2: Fibonacci . . . 35
5.2.1 Código em C- . . . 35
5.2.2 Código intermediário . . . 36
5.2.3 Código assembly . . . 37

5.2.4 Código binário 39
5.3 Exemplo 3: Sort 41
5.3.1 Código intermediário 42
5.3.2 Código assembly 45
5.3.3 Código binário 50

6 CONCLUSÃO 57

REFERÊNCIAS 59

1 Introdução

O rápido avanço da tecnologia na última década certamente elevou a importância da eletrônica digital em um patamar nunca visto antes. É fato que a última geração da humanidade possui, em todo nível social e econômico, podendo até ser de modo indireto, uma certa dependência ou até comodismo proporcionados pela eletrônica digital.

Os sistemas digitais, no mundo atual, se tornaram onipresentes em todos os setores inerentes à sociedade. Como exemplo de alguns desses setores, tem-se a economia, a agricultura, a industria, o entretenimento, a cultura, entre outros.

Segundo Tocci, 2011(1), um sistema digital é uma combinação de dispositivos projetados para manipular informação lógica no formato digital, isto é, informações que podem assumir valores discretos. Esses dispositivos podem ser eletrônicos, mecânicos, magnéticos ou até pneumáticos. Entre os sistemas digitais mais conhecidos, temos os computadores, as calculadoras, os equipamentos de áudio e vídeo e o sistema de telecomunicações.

Um dos componentes presentes na maioria dos sistemas digitais mais complexos é o processador, que poderia ser entendido como sendo o "cérebro"do sistema, local onde todas as operações e tomadas de decisões são realizadas. O processador pode ser manipulado através do chamado Código de Máquina, um código escrito em bits, com palavras de tamanho pré definidos, capazes de indicar ao processador qual operação deve ser realizada, manipulando dados contidos tanto em registradores como em memória.

Sendo assim, o profissional atuante em áreas que se relacionam com a eletrônica digital deve dominar com maestria uma variedade diversificada de conceitos teóricos e práticos sobre sistemas digitais, incluindo conceitos relacionados a processadores e linguagens de programação.

Linguagens de programação são, fundamentalmente, um modo de descrever computações, amigável tanto para pessoas, como para máquinas, a fim de facilitar a comunicação e a manipulação entre essas duas entidades. O mundo atual depende de linguagens de programação em um nível nunca visto antes. Todos os dispositivos eletrônicos utilizados pelas pessoas possuem software escrito em alguma linguagem de programação. Segundo (2), para que um programa escrito em alguma linguagem de programação possa realizar seu papel, primeiro ele precisa ser interpretado e traduzido para um formato que o computador ou o sistema computacional venha a entender, que seria o código de máquina. A entidade responsável por realizar essa tradução se chama compilador, que é, também, um software.

Um dos objetivos deste relatório, além do expor o processo de implementação de um sistema computacional munido de processador e compilador, foi expor esses conceitos

de forma mais realista e prática, e sua realização tem um papel fundamental na formação profissional na área.

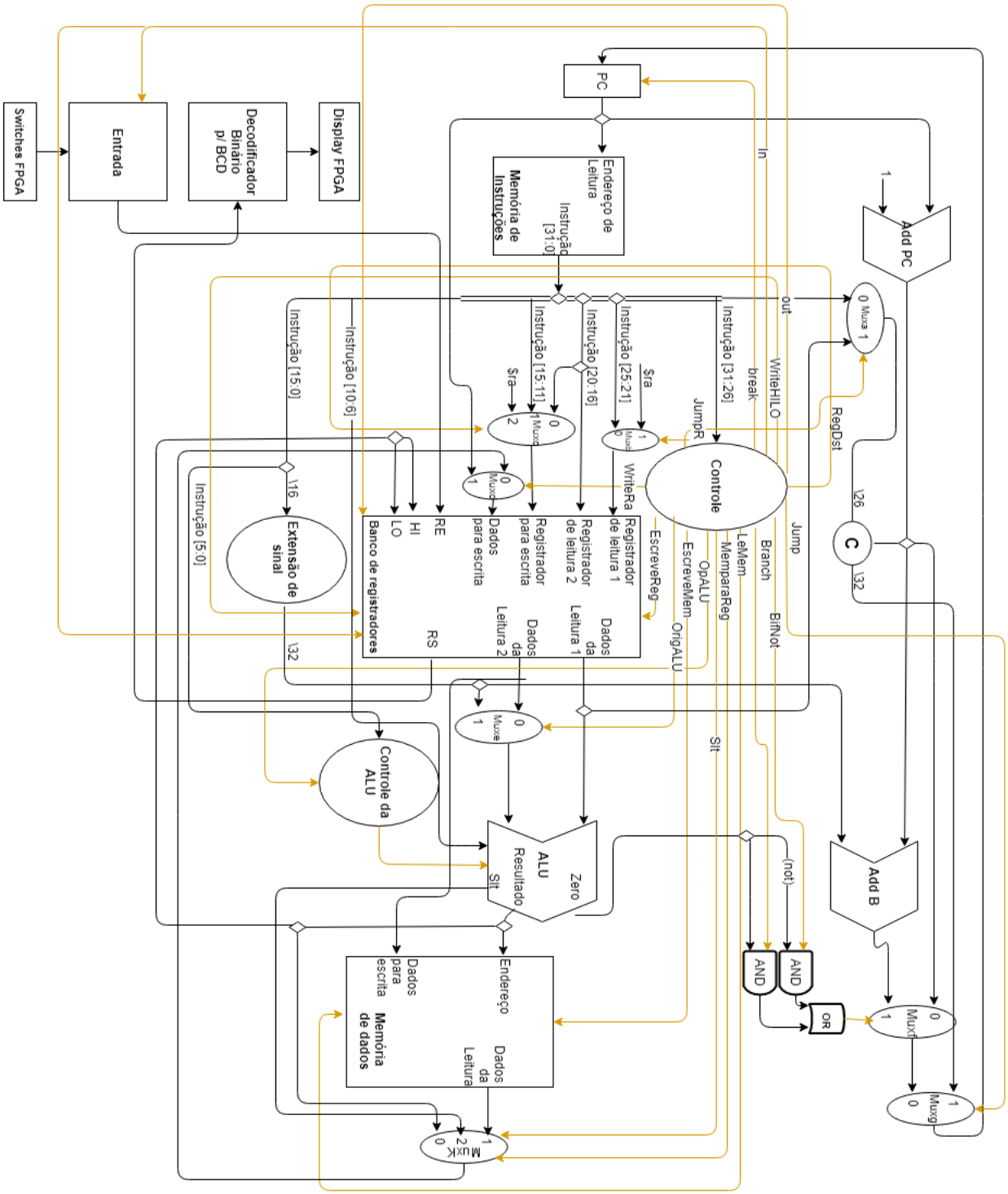
O projeto, portanto, consiste na elaboração de um processador funcional, capaz de realizar um conjunto de operações fundamentais na implementação dos algoritmos mais básicos de lógica de programação, e na elaboração de um compilador, capaz de traduzir a linguagem de programação C-, proposta por (3) na linguagem de máquina nativa do processador desenvolvido.

O capítulo 2 apresenta um breve esclarecimento do processador desenvolvido, apresentando um diagrama em blocos, seguido pela explicação de cada módulo ou componente deste diagrama, também evidenciando o conjunto de instruções utilizado e como ocorreu a organização da memória. Os capítulos 3 e 4 buscam apresentar detalhadamente o compilador desenvolvido, desde a etapa de análise, onde a possibilidade de erros do código compilado é verificada até a etapa de síntese, onde o código de máquina é gerado automaticamente. Por fim, o capítulo 5 irá desenvolver a discussão dos resultados e conclusões do projeto, de um modo geral, apresentando exemplos de código compilado e as principais dificuldades encontradas durante a implementação.

2 O processador

Durante a implementação do processador, houve a elaboração de um diagrama que ilustra todos os módulos ou subsistemas utilizados e a interação entre eles.

Figura 1 – Caminho de Dados, elaborado com a ferramenta <draw.io>



2.1 Módulos do processador

Nesta seção, haverá uma breve descrição dos módulos mais importantes ou evidentes, que mais contribuem para que o processador funcione de modo apropriado.

2.1.1 Unidade de controle

No diagrama, o bloco denotado por "controle"é responsável por controlar o processador como um todo, através de sinais enviados aos outros módulos, fazendo com que operem de modo condizente à instrução corrente. Basicamente, a entrada deste módulo em questão é o código da instrução executada e a saída são esses sinais, ligados nos demais módulos do processador.

2.1.2 Program Counter

No diagrama, o bloco denotado por "PC"é responsável por armazenar o endereço da instrução corrente a ser executada, de modo que um programa executado pelo processador possua uma iteração entre suas operações coerente.

2.1.3 Memória

No diagrama, os blocos denotados por "Memória de instruções"e "Memória de Dados"são responsáveis por armazenar a instruções a serem executadas pelo processador durante a execução de um programa, e os dados utilizados por esses programas, respectivamente.

2.1.4 Banco de Registradores

O banco de registradores é o módulo responsável por controlar e gerenciar o uso dos registradores internos do processador na execução de instruções.

2.1.5 Unidade lógica e aritmética

O módulo denotado por "ALU"(Arithmetic Logic Unit) é responsável por executar todas as operações lógicas e aritméticas, assim como todos os cálculos de endereço para operações que envolvam o manuseio de registradores ou de memória.

2.2 Interação entre os elementos do caminho de dados

Como o caminho de dados ficou bem denso, haverá uma breve explicação para facilitar o entendimento. Uma particularidade desta ilustração é o uso de losangos pequenos

em cima dos fios para indicar uma bifurcação, quando é necessário enviar a mesma informação para dois ou mais módulos distintos. As setas coloridas representam os sinais de controle.

Vale ressaltar que, como a memória vai ser mapeada em palavras de 32 bits, diferente do que ocorre no MIPS, a cada instrução o PC será incrementado de modo unitário.

As instruções do formato **J** ocorrem da seguinte forma: Primeiramente, a Memória de Instruções recebe do registrador PC o Endereço de leitura da instrução a ser executada e a envia para um barramento indicado pelos traços duplos, que contém a instrução toda. Logo depois disso, os 26 bits menos significativos são enviados ao único multiplexador vertical da ilustração, que possui a função de distinguir se a operação de salto se trata de uma instrução "j"ou "jr". Logo após isso, o sinal é concatenado com os 6 bits mais significativos do PC, para formar o endereço do desvio, e, assim, o valor obtido é enviado para o PC.

Para explicar como ocorre uma instrução do tipo **I**, vamos utilizar a instrução "beq". Assim como nas instruções do formato anterior e nas demais, em um primeiro momento, a memória de instruções passa para o barramento denotado pelas barras duplas a instrução a ser executada. No caso de uma instrução "beq", o banco de registradores recebe o endereço dos 2 registradores a serem comparados envia seus respectivos valores para a ALU. Enquanto isso, O campo "Imediato"da instrução é enviado para a unidade de "Extensão de sinal"na parte inferior do diagrama, que possui a finalidade de estender o sinal de 16 bits para 32, afim de possibilitar a soma com o endereço atual do PC, e enviar o resultado para o PC novamente, caso o desvio seja tomado. Em um terceiro momento, a ALU realiza a subtração dos operandos que recebeu e, caso a subtração resulte em 0, significando que os valores são iguais, o sinal de saída "Zero"da ALU é enviado para região superior direita da ilustração, que possui uma lógica que faz com que o desvio seja tomado caso o sinal de saída apropriado seja enviado.

Uma instrução do tipo **R** ocorre da seguinte forma: Após a leitura e a passagem da instrução para o barramento de barras duplas, a unidade de controle faz com que os campos "Registrador de leitura 1"receba o campo "R1"da instrução, "Registrador de leitura 2"receba o campo "R2"da instrução e o campo "Registrador para escrita"receba o campo "Rd"da instrução. Posteriormente, os valores de "R1"e "R2"são enviados para a ALU, que realiza a operação e envia o resultado para o campo "Dados para escrita"que acabam por serem escritos no registrador apontado pelo campo "Rd", presente em "Registrador para escrita"

2.3 Conjunto de instruções escolhido e suas particularidades

Lembrando que os conjunto de instruções adotado segue as características de um conjunto RISC.

Instruções de Operações Aritméticas

- **add**: Soma básica entre 2 valores armazenados em registradores.
- **addi**: Soma básica entre um valor armazenado em um registrador e um imediato. Note que um imediato é um número representado diretamente no código da instrução.
- **sub**: Subtração básica de dois valores armazenados em registradores.
- **subi**: Substração básica de um valor armazenado em um registrador com um imediato.
- **mult**: Multiplicação de dois valores armazenados em registradores. Os 32 bits mais significativos do resultados são armazenados no registrador HI e os 32 menos significativos nos registrador LO.
- **div**: Divisão inteira entre dois valores armazenados em registradores. O resultado da divisão é armazenado no registrador LO e o resto no HI.

Instruções de Transferência de Informação:

- **lw**: Carrega uma palavra de 32 bits da memória para um registrador do banco de registradores.
- **sw**: Salva uma palavra de 32 bits armazenada em um registrador na memória.

Instruções de Deslocamento de bits:

- **sra**: Desloca de modo aritmético um número para a direita.
- **sla**: Desloca de modo aritmético um número para a esquerda.

Vale ressaltar que o deslocamento aritmético, diferente do deslocamento lógico, sempre multiplica por 2 (deslocando para a direita) ou divide por 2 (deslocando para a esquerda) um número binário, mesmo quando o número é representado em complemento de 2. No caso do deslocamento lógico, isso só funciona para números sem sinal.

Instruções de Operações Lógicas e de Comparação:

- **and**: Realiza uma operação de and lógica entre valores armazenados em 2 registradores.

- **andi**: Realiza uma operação de and lógica entre um valor imediato e um valor armazenado em um registrador.
- **or**: Realiza uma operação de or lógica entre valores armazenados em 2 registradores.
- **ori**: Realiza uma operação de or lógica entre um valor imediato e um valor armazenado em um registrador.
- **xor**: Realiza uma operação xor entre valores armazenados em 2 registradores.
- **xori**: Realiza uma operação xor com um valor armazenado em um registrador e um imediato.
- **nor**: Realiza uma operação nor lógica entre valores armazenados em 2 registradores.
- **slt**: Compara o valor de 2 registradores através de uma subtração. Se o valor do primeiro registrador for menor do que o do segundo, um registrador especificado pelo código da instrução recebe o valor 1.
- **slti**: Compara o valor de um registrador com um imediato. Se o valor do registrador for menor do que o imediato, um registrador especificado pelo código da instrução recebe o valor 1.
- **not**: Realiza uma operação de negação com o valor de um registrador.

Instruções de Salto e desvio:

- **j**: Operação que realiza um salto do endereço da instrução atual para um outro endereço da memória de instruções, determinado pelo código da instrução. Desse modo, há a possibilidade de "pular"um uma determinada quantidade de instruções, que acabam deixando de serem executadas. O modo como isso ocorre será explicado quando o diagrama em blocos do processador for apresentado.
- **jr**: Operação que realiza um salto do endereço da instrução atual para o endereço armazenado no registrador $ra.
- **beq**: Compara o valor de 2 registradores. Se forem iguais, um desvio é realizado para o endereço especificado no código da instrução. Um desvio pode ser entendido como um salto que ocorre condicionalmente.
- **bne**: Compara o valor de 2 registradores e realiza um desvio para um endereço especificado pelo código da instrução se esses dois valores forem diferentes.
- **jal**: Realiza a mesma operação da instrução j, porém salva o valor do endereço atual no registrador $ra. Essa instrução é utilizada em conjunto com a instrução jr para possibilitar a criação de funções na memória de instruções.

Vale ressaltar que, caso necessário, existe a possibilidade da inclusão de novas instruções no conjunto trabalhado, do mesmo modo que há a possibilidade da exclusão de instruções já existentes.

2.4 Organização da memória

Como já definido anteriormente, na seção 2.2, a memória foi mapeada em palavras de 32 bits, com o intuito de facilitar e simplificar a implementação do processador e o gerenciamento de memória realizado pelo compilador, que será esclarecido posteriormente.

3 Compilador: Fase de análise

A fase de análise do compilador é responsável pela investigação ou pesquisa por erros no código a ser traduzido, e caso um erro é encontrado, o processo de compilação é interrompido. Esse processo é necessário uma vez que o programador é passivo de cometer erros durante a implementação de algum programa, e a ausência dessa etapa iria comprometer o processo todo, uma vez que a tradução para código máquina de um programa implementado incorretamente iria ocasionar um funcionamento não esperado do processador.

A fase de análise pode ser segmentada em três maiores procedimentos, que detectam diferentes tipos de erros, a análise léxica, sintática e semântica.

3.1 Análise léxica

Segundo (2), a tarefa principal do analisador léxico, a primeira fase de um compilador, é ler os caracteres do programa a ser traduzido, agrupá-los em lexemas e atribuir um token a cada lexema obtido, tendo como saída uma lista de tokens, que vão ser utilizados pela próximo procedimento, a análise sintática.

Um lexema é uma unidade básica de um léxico, podendo ser entendido como uma palavra ou locução. Um token é uma classificação a ser atribuída a algum lexema. No caso do projeto de um compilador, um lexema pode ser classificado em palavra reservada, número, identificador, etc.

Normalmente, o analisador léxico também interage com a tabela de símbolos, uma outra entidade gerada no processo de análise responsável por verificar erros no procedimento da análise semântica, e por auxiliar na geração automática de código. Porém, o presente projeto gera a tabela de símbolos a partir da arvore de análise sintática, outra entidade utilizada pelo compilador, que será esclarecida posteriormente.

O analisador léxico, no caso do compilador implementado, caso um erro seja encontrado, ou seja, um lexema obtido não se encaixar na classificação de nenhum token, deve interromper o processo de compilação e disparar uma mensagem de erro.

3.2 Análise sintática

Segundo (2), o papel do analisador sintático é verificar se a lista ou cadeia de tokens obtida através do analisador léxico corresponde à linguagem gerada pela gramática.

Lembrando que maiores informações sobre os tokens e a gramática da linguagem C- podem ser encontrados em (3).

No caso da lista de tokens não pertencer a gramática da linguagem de programação C-, o analisador sintático deve disparar uma mensagem de erro, e, assim como o analisador léxico, interromper o processo.

É nesta etapa que a árvore de análise sintática é produzida. Isso ocorre conforme a verificação vai se expandindo pelas cláusulas da gramática. Em cada expansão, um novo nó pode ser inserido na árvore, na posição adequada. Ela, em conjunto com a tabela de símbolos, pode ser utilizada nas etapas de análise semântica e de síntese.

3.3 Análise semântica

A análise semântica, responsável por examinar as estruturas geradas pela compilação até o momento, no caso do presente projeto, se utiliza da tabela de símbolos e ocorre enquanto a mesma é construída. Uma vez que a árvore de análise sintática esta completa, junto com o fim da etapa de analise sintática, o processo de construção da tabela de símbolos é iniciado.

O processo de construção da tabela de símbolos ocorre em função de uma iteração sobre a árvore de análise sintática, percorrida em pré-ordem, inserindo os nós iterados na tabela e realizado as verificações apropriadas que caracterizam a análise semântica. Essas verificações identificam erros de:

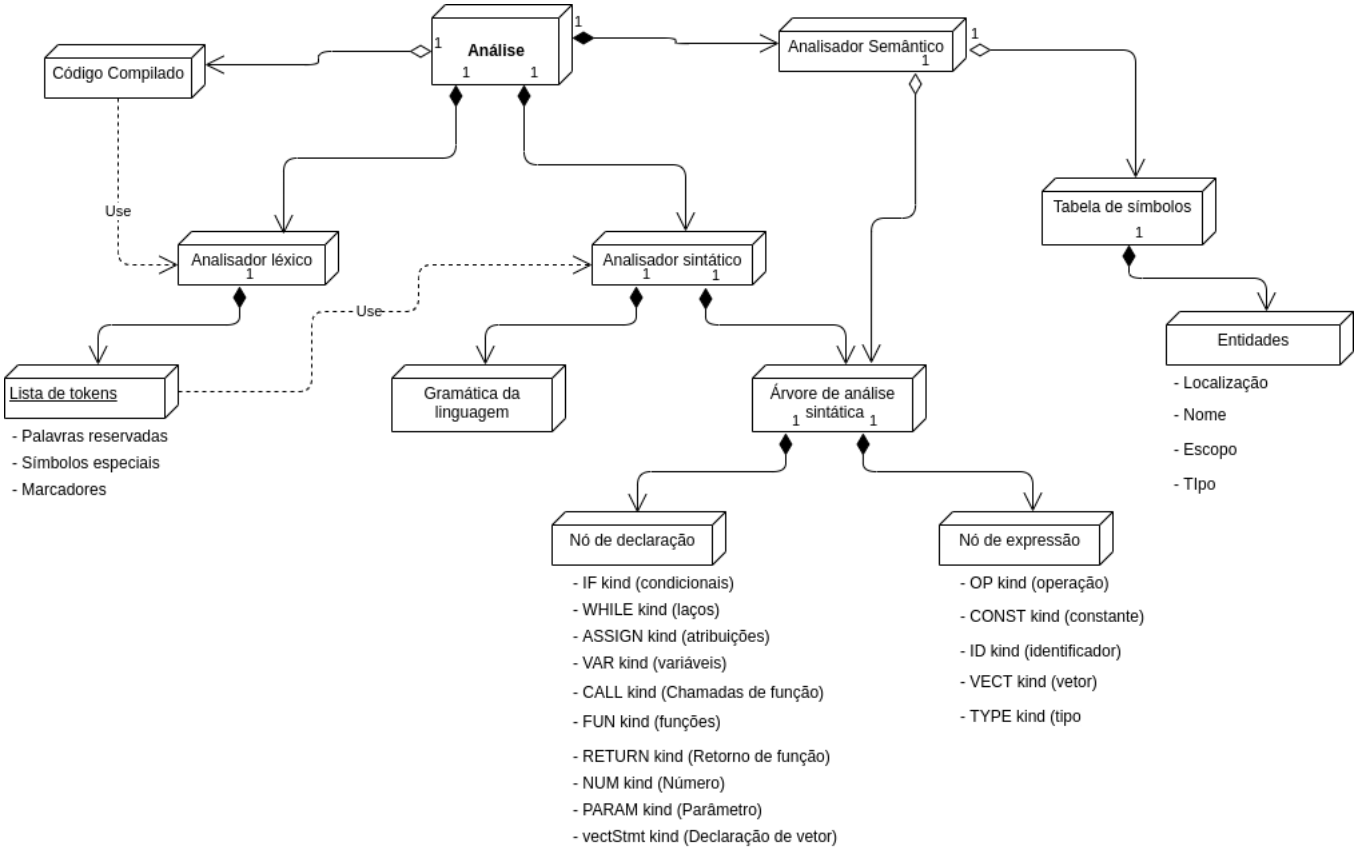
- Variável não declarada no escopo corrente;
- Atribuição inválida;
- Declaração inválida;
- Chamada de função não declarada;
- Função principal(main) não declarada.

Por fim, esta etapa, caso entre em contato com algum erro, diferentemente das anteriores, possui a capacidade de identificar o tipo de erro encontrado, então ao interromper o processo de compilação, em decorrência de um erro encontrado, deve apresentar qual foi o tipo desse erro.

3.4 Modelagem

A fim de facilitar o entendimento de como ocorrem esses procedimentos de análise, assim como eles interagem e se complementam, vamos examinar alguns diagramas:

Figura 2 – Diagrama em bloco do módulo de análise, elaborado com a ferramenta <draw.io>. Fonte: O Autor



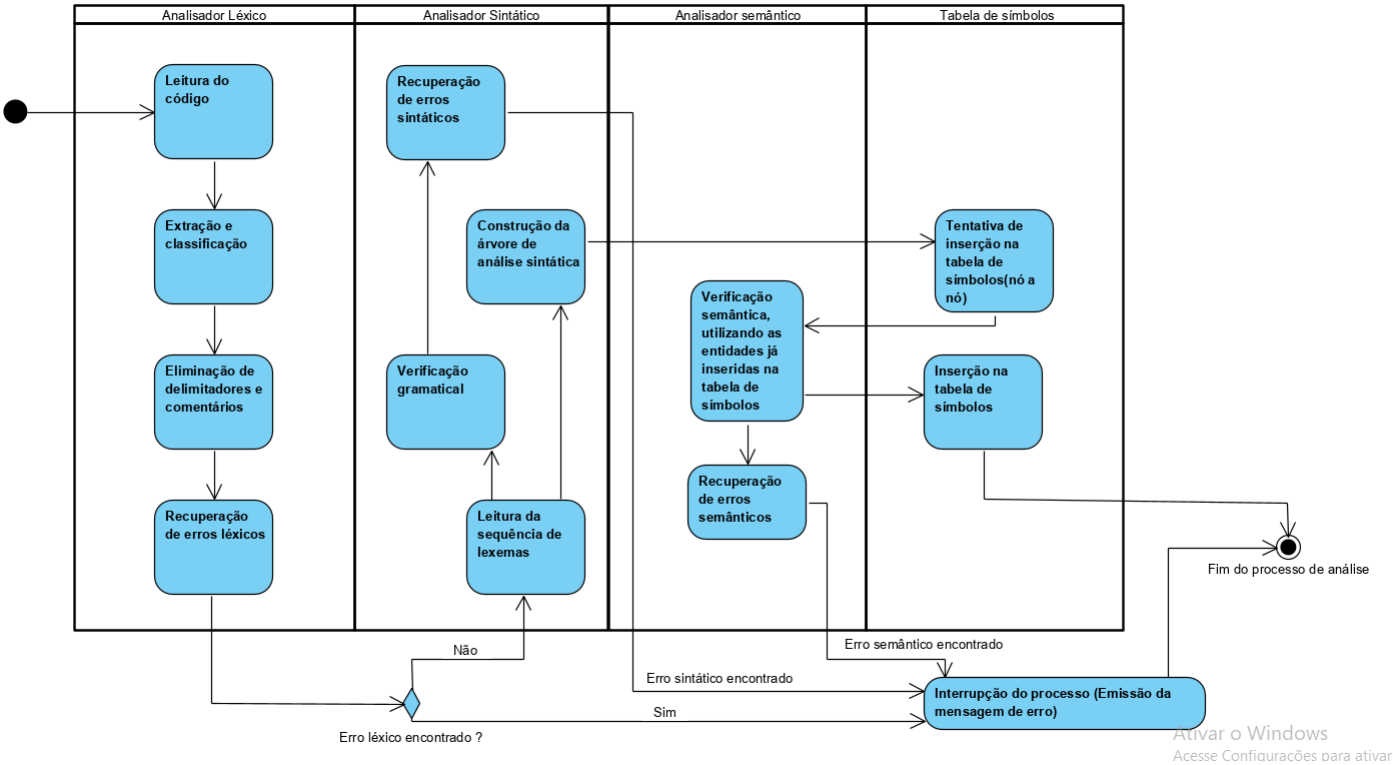
Neste diagrama, fica fácil observar a dependência da etapa de análise com os procedimentos de análise léxica, análise sintática e análise semântica. Note que o analisador léxico, como constatado anteriormente, gera uma lista de tokens, que é utilizada pelo analisador sintático.

O analisador sintático por sua vez, é responsável pela manutenção da árvore de análise sintática, utilizada pelo analisador semântico para realizar inserções na tabela de símbolos e verificar a integridade semântica do programa compilado.

Note que todas as estruturas de dados utilizadas estão presentes no diagrama como entidades com atributos. A lista de tokens, por exemplo, pode conter tokens que representam palavras reservadas, símbolos especiais, e marcadores. A árvore de análise sintática possui dois tipos principais de nós, que representam uma declaração ou uma expressão, estes que podem receberem outras classificações, denotadas pelos atributos dos blocos de nó de declaração e nó de expressão.

A tabela de símbolos, por sua vez, como se pode observar no diagrama, é constituída de entidades ou elementos do código compilado munidos de nome, localização, escopo e tipo.

Figura 3 – Diagrama de atividades do módulo de análise, elaborado com a ferramenta <draw.io>. Fonte: O Autor



Este diagrama busca descrever o modo com o qual toda a etapa de análise é realizado, dividindo os processos e etapas já apresentados em subetapas, e destacando a relação e a interação entre elas.

Note que o processo começa com a atuação do analisador léxico, como já descrito anteriormente, representado no diagrama pela primeira coluna. O analisador léxico, por sua vez, realiza a leitura do código compilado, e, a partir dessa leitura, ocorre a extração e classificação dos lexemas, junto com a eliminação de delimitadores e comentários, processo que normalmente é realizado em uma etapa de pré-compilação.

Após esse primeiro momento, há a verificação se erros léxicos foram encontrados e, caso negativo, inicia-se a etapa de análise sintática. Caso um erro seja encontrado, o processo é interrompido e finalizado.

O analisador semântico, por sua vez, realiza a leitura da lista de lexemas gerada pela etapa anterior, e faz a verificação gramatical e a construção da árvore de análise sintática de modo simultâneo. A verificação da gramática, caso encontre erros sintáticos, dispara uma mensagem de erro e interrompe o processo, como pode ser observado no diagrama. A construção da árvore de análise sintática, uma vez finalizada, segue para a construção da tabela de símbolos, inserindo um nó por vez. O processo de inserção na tabela de símbolos é munido de verificações quanto a possíveis erros de natureza semântica, como indica o diagrama. E caso uma dessas verificações dispare um erro semântico, o

processo de compilação é interrompido.

Se erros semânticos não são encontrados, a construção da tabela de símbolos é finalizada junto com o processo inteiro de análise, e a compilação segue para a etapa de síntese.

Vale ressaltar que uma das decisões do projeto foi utilizar as ferramentas flex e bison nas etapas de análise léxica e análise sintática, respectivamente.

4 Fase de síntese

4.1 Modelagem

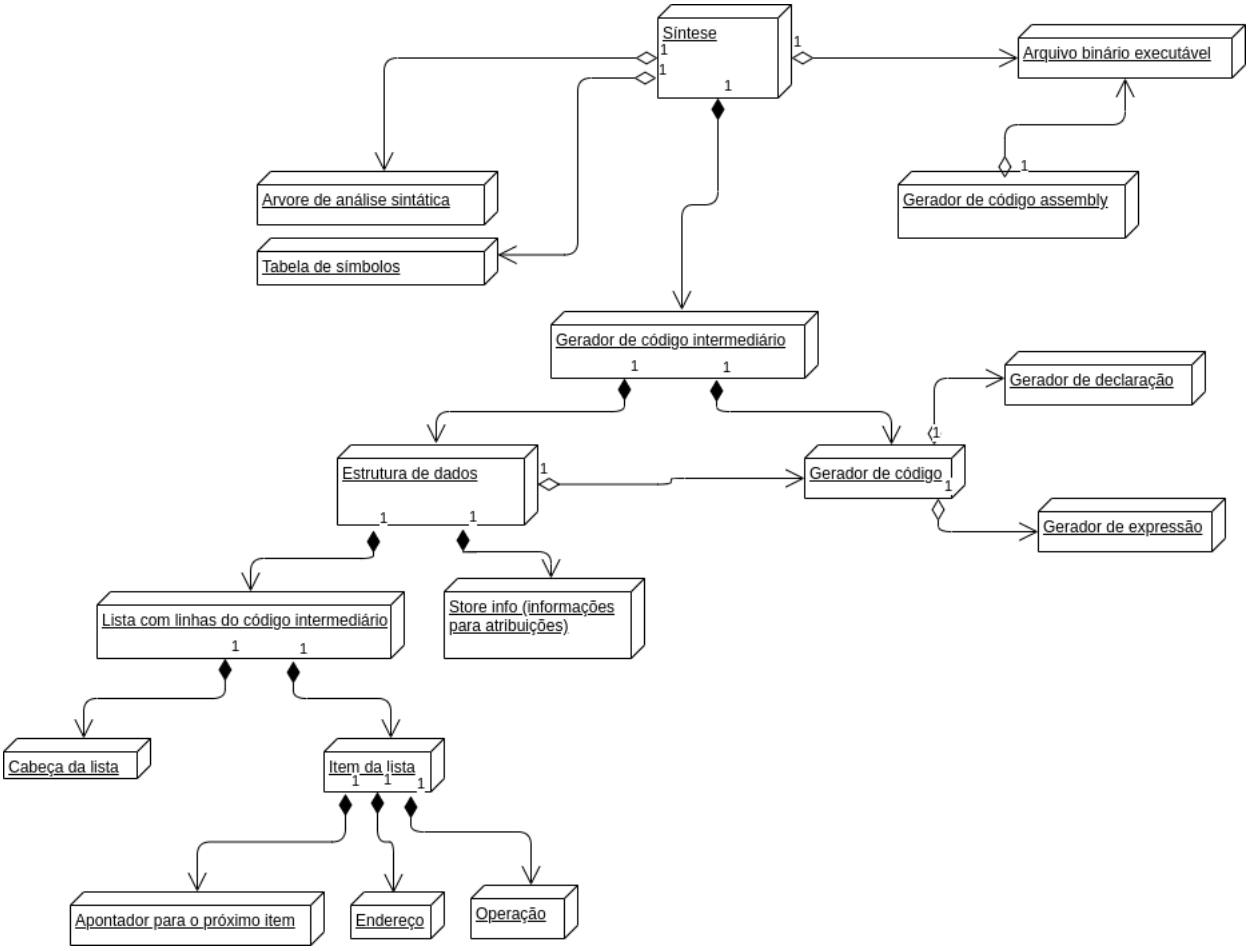
Em um primeiro momento, deve haver um certo planejamento e discussão a respeito dos requisitos necessários na fase de síntese, antes da implementação.

Como decisões de projeto, optou-se por subdividir a etapa de síntese de código em 3 subetapas, a fim de facilitar e simplificar a implementação e a organização do gerador de código. A primeira etapa consiste na geração de um código intermediário, utilizando a árvore de análise sintática gerada na etapa de análise, com a finalidade de facilitar a implementação do código assembly, que seria a segunda etapa. O código intermediário atua de fato como um facilitador, suas operações não possuem tanta importância e são particulares do projeto. Já o assembly é um tipo de código mais consolidado na literatura.

A última subetapa é a geração de código binário, a partir do código assembly. Esta etapa não possui muitos requisitos e particularidades, e é direta, não há a necessidade de manipular estruturas maiores. Atua basicamente como um decodificador do codigo assembly, transformando os labels, registradores, imediatos e instruções em código de máquina, de modo que o processador possa ler e executar o programa.

A figura 4 ilustra um diagrama desenvolvido a fim de facilitar o entendimento da etapa completa de síntese.

Figura 4 – Diagrama em bloco do módulo de síntese, elaborado com a ferramenta <draw.io>. Fonte: O Autor



Note que a árvore de análise sintática e a tabela de símbolos são duas entidades construídas na etapa de análise, porém estão presentes na etapa de síntese, por serem estruturas de dados utilizadas na geração de código.

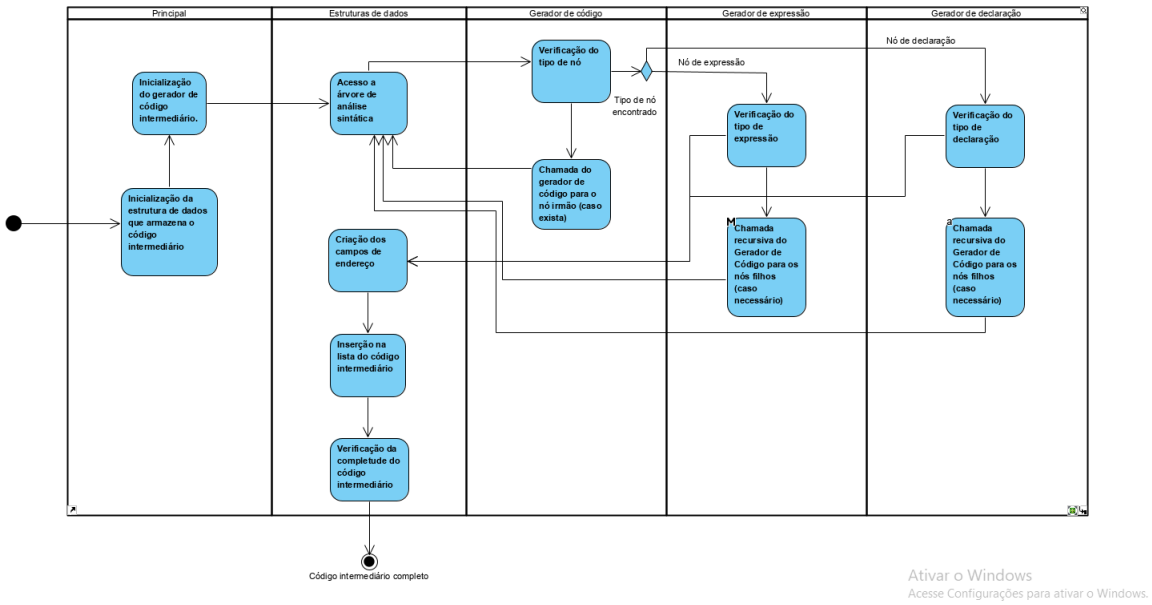
O gerador de código intermediário necessitou de um nível de complexidade maior, como se pode observar com o diagrama, para que a leitura recursiva da árvore sintática fosse realizada apropriadamente. Vale ressaltar que o código intermediário foi gerado em forma de lista, com cada item correspondendo a uma operação, e essa etapa foi subdividida de acordo com os tipos de nó da arvore sintática.

O código assembly não necessitou de tanta complexidade quanto a estrutura de dados utilizada, uma vez que a lista responsável por carregar o código intermediário já providenciou todos os recursos necessários. O modo de geração do código assembly ocorreu em função de um laço que percorre toda a lista de código intermediário, e será esclarecido posteriormente.

Por último, vale ressaltar que optou-se por implementar o gerador de código binário em python. Uma vez que essa etapa decodifica diretamente o assembly em linguagem de

máquina, o uso dessa tecnologia facilitou o desenvolvimento.

Figura 5 – Diagrama de atividades do módulo de síntese, elaborado com a ferramenta <draw.io>. Fonte: O Autor



Neste diagrama, fica mais claro de entender qual é a função de cada etapa na síntese de código, e como elas interagem entre si. No caso, essas etapas são entidades e estão sendo representadas pelas colunas. Seguindo o fluxo denotado pelo segmentos das setas entre as atividades, em um primeiro momento, no início do processo de síntese como pode se observar no diagrama, há a inicialização da estrutura de dados que irá armazenar o código intermediário, junto com a inicialização do gerador de código intermediário.

Note que a coluna denotada por estrutura de dados comporta as ações de todas as estruturas de dados utilizadas na elaboração de código: A árvore sintática, os campos de endereço, e a lista de código intermediário.

Como dito anteriormente, a árvore de análise sintática é percorrida durante o processo, então primeiro tem-se um acesso a árvore sintática, identificando o primeiro nó a ser transformado em código intermediário. Em um primeiro momento, o gerador de código é responsável por verificar o tipo de nó iterado, e terá um tratamento diferente caso seja um nó de declaração ou de expressão.

Note que o gerador de código, além da verificação do tipo de nó, realiza a chamada recursiva de si mesmo para o nó irmão do nó corrente, caso exista, realizando um novo acesso na arvore sintática, e dando inicio ao procedimento novamente. Isso acontece até que a arvore sintática seja inteiramente percorrida.

Os geradores de expressão e de declaração realizam a primeira verificação, como pode-se observar no diagrama, e partir dessa verificação, um ou mais itens de código

intermediário serão criados, com seus respectivos campos de endereço, e o procedimento de geração de código é chamado recursivamente para os nós filhos, dependendo do tipo de nó corrente examinado.

Após o percorrimento de toda a árvore, a completude do código intermediário é verificada e se inicia o processo de geração do código assembly.

Como o processo de geração de assembly é realizada a partir de uma iteração sobre a lista gerada pelo código intermediário, optou-se por não entrar em maiores detalhes por meio do diagrama.

4.2 Geração de código intermediário

Como já constatado anteriormente, a etapa de geração do código intermediário é o início da fase de síntese do compilador. Não é uma etapa obrigatória mas sua implementação facilita o trabalho das próximas etapas. Sua unidade principal é a chamada quádrupla, e cada item da lista utilizada como representação do código intermediário descreve uma quádrupla. Uma quádrupla, por sua vez, possui no máximo 4 elementos: a operação a ser realizada e três endereços para a realização dessa operação. Esse modelo é definido por (3).

É importante ressaltar que o gerador de código intermediário tem como entrada a árvore de análise sintática, e como saída a lista de código intermediário, que será utilizada pelo gerador de código assembly. A tabela abaixo apresenta todas as operações que o código intermediário pode vir a utilizar.

Tabela 1 – Operações realizadas pelo código intermediário

Operação	Descrição
ADD	Adição
SUB	Subtração
MULT	Multiplicação
DIV	Divisão
LT	Menor
EQ	Igual
LET	Menor ou igual
GT	Maior
GET	Maior ou igual
NOTEQ	Diferente
AND	e lógico
OR	ou lógico
ASSIGN	Atribuição
ALLOC	Alocação
IMMED	Imediato
LOAD	Carregar da memória
STORE	Armazenar na memória
GOTO	Ir para
IFF	Se falso, vai para
RET	Retorno
FUN	Início de função
END	Fim de função
PARAM	Parâmetro de função
CALL	Chamada de função
ARG	Parâmetro de chamada
LAB	Rótulo
HLT	Parada
INPUT	Entrada de dados
OUTPUT	Saída de dados

4.3 Geração de código assembly

A próxima etapa é a geração do código assembly. Neste momento, cada quádrupla gerada pela etapa anterior é traduzida, de modo iterativo, em uma ou mais instruções do conjunto escolhido pro processador desenvolvido, como já foi comentado anteriormente. Todas as quádruplas possuem um processo particular para se transformarem em instruções. Então, a implementação consistiu basicamente em um laço iterando sobre todos os itens da lista que contém o código intermediário, realizando um processo diferente para cada operação, por meio do uso do laço condicional switch.

Vale ressaltar que, ao invés de endereços, como no código intermediário, o assembly utiliza os registradores nativos do processador. O banco de registradores do processador desenvolvido possui um total de 32, sendo 17 deles utilizados com temporários, 6 utilizados como passagem de argumentos para funções, um deles responsável por sempre armazenar o valor 0, um registrador responsável por controlar o endereço do topo da pilha na memória que armazenará variáveis globais, um responsável por armazenar o endereço do topo da

pilha que armazenará variáveis locais, um responsável por armazenar retorno de funções, um responsável por armazenar dados fornecidos pelos dispositivos de entrada e saída do processador, um responsável por armazenar endereços de retorno de funções, e por último, dois registradores responsáveis por armazenar resultados de multiplicações e quociente e resto de divisões.

O código assembly também é armazenado por meio de uma lista, mas foi implementado uma rotina para imprimir o código assembly em um arquivo, facilitando o modo com o qual o gerador de código binário executável irá operar.

4.4 Geração de código binário

Como já dito anteriormente, está é a última fase da etapa de síntese do compilador. Seu objetivo é traduzir o código assembly gerado para o código binário correspondente da máquia alvo. No caso, do processador desenvolvido. Como essa etapa não exige a implementação de uma nova estrutura, optou-se por utilizar python em sua implementação. E por isso, se tornou necessário o carregamento do código assembly em um arquivo.

Vale ressaltar que a verificação da corretude e da totalidade dessa etapa é mais trabalhosa, uma vez que é necessário verificar o código binário gerado.

O script desenvolvido em python que implementa essa etapa itera sobre todas as linhas do arquivo que contém o código assembly, transcrevendo opcodes, registradores, imediatos e convertendo os labels para seu valor correspondente de memória, utilizando o número da linha do arquivo assembly como referência.

4.5 Gerenciamento de memória

O gerenciamento de memória adotado ocorre através da simulação de uma pilha virtual na memória, através do uso dos registradores $gp(global pointer) e $sp(stack pointer) que armazenam endereços da memória, o primeiro armazena o endereço em que começa a sequência de armazenamento das variáveis globais, e o segundo armazena o endereço do topo da pilha virtual. Operações de inserir e retirar itens da pilha são realizadas por meio deste registrador.

Sendo assim, a memória ficou organizada da seguinte forma:

- O registrador $gp sempre possuirá valor 0 até o presente momento, indicando que as variáveis globais começam a aparecerem a partir do endereço de memória 0, e que a parcela da memória de dados reservada ao programa atual em execução começa no endereço 0.

- O registrador $sp recebe o endereço de memória da última variável global do programa, fazendo com que, para que um item seja inserido na pilha simulada, sp seja incrementado de uma unidade.

5 Exemplos

Neste capítulo, vamos examinar alguns exemplos do gerador de código funcionando, ilustrando o código gerado em cada etapa do processo de síntese. A complexidade dos exemplos aumenta conforme se dá o segmento dos exemplos.

5.1 Exemplo 1: GCD

Esse exemplo calcula o maior divisor comum de um número passado como entrada.

5.1.1 Código em C-

```
int gcd (int u, int v){
        if ( v == 0 ) return u;
        else return gcd(v, u-u/v*v);
}

void main(void){
        int x; int y;
        x = input(); y=input();
        output(gcd(x,y));
}
```

5.1.2 Código intermediário

```
/*
<fun: int, gcd, -->
<param: int, u, gcd>
<param: int, v, gcd>
<load: $t0, v, -->
<immed: $t1, 0, -->
<eq: $t2, $t0, $t1>
<iff: $t2, label0, -->
<load: $t3, u, -->
<ret: $t3, --, -->
<goto: label1, --, -->
<lab: label0, --, -->
<load: $t4, v, -->
<arg: $t4, --, -->
<load: $t5, u, -->
<load: $t6, u, -->
<load: $t7, v, -->
<div: $t8, $t6, $t7>
<load: $t9, v, -->
<mult: $t10, $t8, $t9>
<sub: $t11, $t5, $t10>
<arg: $t11, --, -->
<call: $t12, gcd, 2>
<ret: $t12, --, -->
<lab: label1, --, -->
<end: gcd, --, -->
```

```
<fun: void, main, -->
<alloc: x, main, 1>
<alloc: y, main, 1>
<load: $t13, x, -->
<input: $t14, input, 0>
<assign: $t13, $t14, -->
<store: x, $t13, $zero>
<load: $t15, y, -->
<input: $t16, input, 0>
<assign: $t15, $t16, -->
<store: y, $t15, $zero>
<load: $t17, x, -->
<arg: $t17, --, -->
<load: $t18, y, -->
<arg: $t18, --, -->
<call: $t19, gcd, 2>
<arg: $t19, --, -->
<output: $t20, output, 1>
<end: main, --, -->
*/
```

5.1.3 Código assembly

```
/*      addi $sp $zero -1
        addi $gp $zero 0
        jal main
        break
gcd
        addi $sp $sp 1
        sw $sp $a0 0
        addi $sp $sp 1
        sw $sp $a1 0
        subi $t0 $sp 0
        lw $t0 $t0 0
        addi $t1 $zero 0
        eq $t2 $t0 $t1
        beq $t2 $zero label0
        subi $t3 $sp 1
        lw $t3 $t3 0
        addi $ret $t3 0
        j label1
label0
        subi $t4 $sp 0
        lw $t4 $t4 0
        addi $a0 $t4 0
        subi $t5 $sp 1
        lw $t5 $t5 0
        subi $t6 $sp 1
        lw $t6 $t6 0
        subi $t7 $sp 0
        lw $t7 $t7 0
        div   $t6 $t7
        add $t8 $LO $zero
        subi $t9 $sp 0
        lw $t9 $t9 0
        mult   $t8 $t9
        add $t10 $LO $zero
        sub $t11 $t5 $t10
        addi $a1 $t11 0
```

```
        addi $sp $sp 1
        sw $sp $t3 0
        addi $sp $sp 1
        sw $sp $t4 0
        addi $sp $sp 1
        sw $sp $t11 0
        addi $sp $sp 1
        sw $sp $ra 0
        jal gcd
        lw $ra $sp 0
        subi $sp $sp 1
        lw $t11 $sp 0
        subi $sp $sp 1
        lw $t4 $sp 0
        subi $sp $sp 1
        lw $t3 $sp 0
        subi $sp $sp 1
        add $t12 $ret $zero
        addi $ret $t12 0
label1
        subi $sp $sp 2
        jr
main
        addi $sp $sp 1
        addi $sp $sp 1
        subi $t13 $sp 1
        lw $t13 $t13 0
        in
        addi $t14 $RE 0
        add $t13 $t14 $zero
        subi $sp $sp 1
        sw $sp $t13 0
        addi $sp $sp 1
        subi $t15 $sp 0
        lw $t15 $t15 0
        in
        addi $t16 $RE 0
        add $t15 $t16 $zero
        subi $sp $sp 0
        sw $sp $t15 0
        addi $sp $sp 0
        subi $t0 $sp 1
        lw $t0 $t0 0
        addi $a0 $t0 0
        subi $t1 $sp 0
        lw $t1 $t1 0
        addi $a1 $t1 0
        addi $sp $sp 1
        sw $sp $t0 0
        addi $sp $sp 1
        sw $sp $t1 0
        addi $sp $sp 1
        sw $sp $ra 0
        jal gcd
        lw $ra $sp 0
        subi $sp $sp 1
        lw $t1 $sp 0
        subi $sp $sp 1
        lw $t0 $sp 0
        subi $sp $sp 1
```

```
        add $t2 $ret $zero
        addi $a0 $t2 0
        out $a0 $zero
        subi $sp $sp 2
        jr
*/
```

5.1.4 Código binário

```
/* C digo bin rio gerado automaticamente
mem_instrucao[0] = { 6'd2, 5'd26, 5'd25, -16'd1 }    //addi
mem_instrucao[1] = { 6'd2, 5'd26, 5'd27, 16'd0 }    //addi
mem_instrucao[2] = { 6'd27, 26'd58 }          //jal main
mem_instrucao[3] = { 6'd28, 26'd0 }   //break
mem_instrucao[4] = { 32'd0 }     //gcd
mem_instrucao[5] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[6] = { 6'd9, 5'd17, 5'd25, 16'd0 }      //sw
mem_instrucao[7] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[8] = { 6'd9, 5'd18, 5'd25, 16'd0 }      //sw
mem_instrucao[9] = { 6'd4, 5'd25, 5'd0, 16'd0 }    //subi
mem_instrucao[10] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
mem_instrucao[11] = { 6'd2, 5'd26, 5'd1, 16'd0 }    //addi
mem_instrucao[12] = { 6'd19, 5'd0, 5'd1, 5'd2, 11'd0 }      //eq
mem_instrucao[13] = { 6'd25, 5'd2, 5'd26, 16'd5 }   //beq label0
mem_instrucao[14] = { 6'd4, 5'd25, 5'd3, 16'd1 }    //subi
mem_instrucao[15] = { 6'd8, 5'd3, 5'd3, 16'd0 }      //lw
mem_instrucao[16] = { 6'd2, 5'd3, 5'd24, 16'd0 }    //addi
mem_instrucao[17] = { 6'd23, 26'd55 }          //j label1
mem_instrucao[18] = { 32'd0 }  //label0
mem_instrucao[19] = { 6'd4, 5'd25, 5'd4, 16'd0 }    //subi
mem_instrucao[20] = { 6'd8, 5'd4, 5'd4, 16'd0 }      //lw
mem_instrucao[21] = { 6'd2, 5'd4, 5'd17, 16'd0 }    //addi
mem_instrucao[22] = { 6'd4, 5'd25, 5'd5, 16'd1 }    //subi
mem_instrucao[23] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
mem_instrucao[24] = { 6'd4, 5'd25, 5'd6, 16'd1 }    //subi
mem_instrucao[25] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
mem_instrucao[26] = { 6'd4, 5'd25, 5'd7, 16'd0 }    //subi
mem_instrucao[27] = { 6'd8, 5'd7, 5'd7, 16'd0 }      //lw
mem_instrucao[28] = { 6'd7, 5'd6, 5'd7, 16'd0 }     //div
mem_instrucao[29] = { 6'd1, 5'd29, 5'd26, 5'd8, 11'd0 }     //add
mem_instrucao[30] = { 6'd4, 5'd25, 5'd9, 16'd0 }    //subi
mem_instrucao[31] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
mem_instrucao[32] = { 6'd5, 5'd8, 5'd9, 16'd0 }    //mult
mem_instrucao[33] = { 6'd1, 5'd29, 5'd26, 5'd10, 11'd0 }     //add
mem_instrucao[34] = { 6'd3, 5'd5, 5'd10, 5'd11, 11'd0 }     //sub
mem_instrucao[35] = { 6'd2, 5'd11, 5'd18, 16'd0 }    //addi
mem_instrucao[36] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[37] = { 6'd9, 5'd3, 5'd25, 16'd0 }      //sw
mem_instrucao[38] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[39] = { 6'd9, 5'd4, 5'd25, 16'd0 }      //sw
mem_instrucao[40] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[41] = { 6'd9, 5'd11, 5'd25, 16'd0 }      //sw
mem_instrucao[42] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[43] = { 6'd9, 5'd31, 5'd25, 16'd0 }      //sw
mem_instrucao[44] = { 6'd27, 26'd4 }           //jal gcd
mem_instrucao[45] = { 6'd8, 5'd25, 5'd31, 16'd0 }      //lw
mem_instrucao[46] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[47] = { 6'd8, 5'd25, 5'd11, 16'd0 }      //lw
mem_instrucao[48] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
```

```
mem_instrucao[49] = { 6'd8, 5'd25, 5'd4, 16'd0 }      //lw
mem_instrucao[50] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[51] = { 6'd8, 5'd25, 5'd3, 16'd0 }      //lw
mem_instrucao[52] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[53] = { 6'd1, 5'd24, 5'd26, 5'd12, 11'd0 }     //add
mem_instrucao[54] = { 6'd2, 5'd12, 5'd24, 16'd0 }    //addi
mem_instrucao[55] = { 32'd0 }  //label1
mem_instrucao[56] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
mem_instrucao[57] = { 6'd24, 26'd0 }      //jr
mem_instrucao[58] = { 32'd0 }    //main
mem_instrucao[59] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[60] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[61] = { 6'd4, 5'd25, 5'd13, 16'd1 }    //subi
mem_instrucao[62] = { 6'd8, 5'd13, 5'd13, 16'd0 }      //lw
mem_instrucao[63] = { 6'd30, 26'd0 }      //in
mem_instrucao[64] = { 6'd2, 5'd28, 5'd14, 16'd0 }    //addi
mem_instrucao[65] = { 6'd1, 5'd14, 5'd26, 5'd13, 11'd0 }     //add
mem_instrucao[66] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[67] = { 6'd9, 5'd13, 5'd25, 16'd0 }      //sw
mem_instrucao[68] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[69] = { 6'd4, 5'd25, 5'd15, 16'd0 }    //subi
mem_instrucao[70] = { 6'd8, 5'd15, 5'd15, 16'd0 }      //lw
mem_instrucao[71] = { 6'd30, 26'd0 }      //in
mem_instrucao[72] = { 6'd2, 5'd28, 5'd16, 16'd0 }    //addi
mem_instrucao[73] = { 6'd1, 5'd16, 5'd26, 5'd15, 11'd0 }     //add
mem_instrucao[74] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[75] = { 6'd9, 5'd15, 5'd25, 16'd0 }      //sw
mem_instrucao[76] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[77] = { 6'd4, 5'd25, 5'd0, 16'd1 }    //subi
mem_instrucao[78] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
mem_instrucao[79] = { 6'd2, 5'd0, 5'd17, 16'd0 }    //addi
mem_instrucao[80] = { 6'd4, 5'd25, 5'd1, 16'd0 }    //subi
mem_instrucao[81] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
mem_instrucao[82] = { 6'd2, 5'd1, 5'd18, 16'd0 }    //addi
mem_instrucao[83] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[84] = { 6'd9, 5'd0, 5'd25, 16'd0 }      //sw
mem_instrucao[85] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[86] = { 6'd9, 5'd1, 5'd25, 16'd0 }      //sw
mem_instrucao[87] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[88] = { 6'd9, 5'd31, 5'd25, 16'd0 }      //sw
mem_instrucao[89] = { 6'd27, 26'd4 }            //jal gcd
mem_instrucao[90] = { 6'd8, 5'd25, 5'd31, 16'd0 }      //lw
mem_instrucao[91] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[92] = { 6'd8, 5'd25, 5'd1, 16'd0 }      //lw
mem_instrucao[93] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[94] = { 6'd8, 5'd25, 5'd0, 16'd0 }      //lw
mem_instrucao[95] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[96] = { 6'd1, 5'd24, 5'd26, 5'd2, 11'd0 }     //add
mem_instrucao[97] = { 6'd2, 5'd2, 5'd17, 16'd0 }    //addi
mem_instrucao[98] = { 6'd29, 5'd17, 5'd26, 16'd0 }     //out
mem_instrucao[99] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
mem_instrucao[100] = { 6'd24, 26'd0 }      //jr
*/
```

5.2 Exemplo 2: Fibonacci

5.2.1 Código em C-

```
1
2
3  int fibonacci(int n){
4          int c; int next;
5          int first; int second;
6          c = 0;
7          while(c <= n){
8                  if(c <= 1){
9                          next = c;
10                 }else{
11                         next = first + second;
12                         first = second;
13                         second = next;
14                 }
15                 c = c + 1;
16         }
17         return next;
18 }
19
20 void main(void){
21         int n;
22         n = input();
23         output(fibonacci(n));
24 }
```

5.2.2 Código intermediário

```
1  /*
2  <fun: int, fibonacci, -->
3  <param: int, n, fibonacci>
4  <alloc: c, fibonacci, 1>
5  <alloc: next, fibonacci, 1>
6  <alloc: first, fibonacci, 1>
7  <alloc: second, fibonacci, 1>
8  <load: $t0, c, -->
9  <immed: $t1, 0, -->
10 <assign: $t0, $t1, -->
11 <store: c, $t0, $zero>
12 <lab: label2, --, -->
13 <load: $t2, c, -->
14 <load: $t3, n, -->
15 <let: $t4, $t2, $t3>
16 <iff: $t4, label3, -->
17 <load: $t5, c, -->
18 <immed: $t6, 1, -->
19 <let: $t7, $t5, $t6>
20 <iff: $t7, label0, -->
21 <load: $t8, next, -->
22 <load: $t9, c, -->
23 <assign: $t8, $t9, -->
24 <store: next, $t8, $zero>
25 <goto: label1, --, -->
26 <lab: label0, --, -->
27 <load: $t10, next, -->
28 <load: $t11, first, -->
29 <load: $t12, second, -->
30 <add: $t13, $t11, $t12>
31 <assign: $t10, $t13, -->
32 <store: next, $t10, $zero>
```

```
<load: $t14, first, -->
<load: $t15, second, -->
<assign: $t14, $t15, -->
<store: first, $t14, $zero>
<load: $t16, second, -->
<load: $t17, next, -->
<assign: $t16, $t17, -->
<store: second, $t16, $zero>
<lab: label1, --, -->
<load: $t18, c, -->
<load: $t19, c, -->
<immed: $t20, 1, -->
<add: $t21, $t19, $t20>
<assign: $t18, $t21, -->
<store: c, $t18, $zero>
<goto: label2, --, -->
<lab: label3, --, -->
<load: $t22, next, -->
<ret: $t22, --, -->
<end: fibonacci, --, -->
<fun: void, main, -->
<alloc: n, main, 1>
<load: $t23, n, -->
<input: $t24, input, 0>
<assign: $t23, $t24, -->
<store: n, $t23, $zero>
<load: $t25, n, -->
<arg: $t25, --, -->
<call: $t26, fibonacci, 1>
<arg: $t26, --, -->
<output: $t27, output, 1>
<end: main, --, -->
*/
```

5.2.3 Código assembly

```
/*      addi $sp $zero -1
        addi $gp $zero 0
        jal main
        break
fibonacci
        addi $sp $sp 1
        sw $sp $a0 0
        addi $sp $sp 1
        addi $sp $sp 1
        addi $sp $sp 1
        addi $sp $sp 1
        subi $t0 $sp 3
        lw $t0 $t0 0
        addi $t1 $zero 0
        add $t0 $t1 $zero
        subi $sp $sp 3
        sw $sp $t0 0
        addi $sp $sp 3
label2
        subi $t2 $sp 3
        lw $t2 $t2 0
        subi $t3 $sp 4
        lw $t3 $t3 0
```

```
24          cmp $t4 $t2 $t3
25          addi $t2 $zero 2
26          eq $t4 $t4 $t2
27          not $t4 $t4
28          beq $t4 $zero label3
29          subi $t5 $sp 3
30          lw $t5 $t5 0
31          addi $t6 $zero 1
32          cmp $t7 $t5 $t6
33          addi $t5 $zero 2
34          eq $t7 $t7 $t5
35          not $t7 $t7
36          beq $t7 $zero label0
37          subi $t8 $sp 2
38          lw $t8 $t8 0
39          subi $t9 $sp 3
40          lw $t9 $t9 0
41          add $t8 $t9 $zero
42          subi $sp $sp 2
43          sw $sp $t8 0
44          addi $sp $sp 2
45          j label1
46  label0
47          subi $t10 $sp 2
48          lw $t10 $t10 0
49          subi $t11 $sp 1
50          lw $t11 $t11 0
51          subi $t12 $sp 0
52          lw $t12 $t12 0
53          add $t13 $t11 $t12
54          add $t10 $t13 $zero
55          subi $sp $sp 2
56          sw $sp $t10 0
57          addi $sp $sp 2
58          subi $t14 $sp 1
59          lw $t14 $t14 0
60          subi $t15 $sp 0
61          lw $t15 $t15 0
62          add $t14 $t15 $zero
63          subi $sp $sp 1
64          sw $sp $t14 0
65          addi $sp $sp 1
66          subi $t16 $sp 0
67          lw $t16 $t16 0
68          subi $t0 $sp 2
69          lw $t0 $t0 0
70          add $t16 $t0 $zero
71          subi $sp $sp 0
72          sw $sp $t16 0
73          addi $sp $sp 0
74  label1
75          subi $t1 $sp 3
76          lw $t1 $t1 0
77          subi $t2 $sp 3
78          lw $t2 $t2 0
79          addi $t3 $zero 1
80          add $t4 $t2 $t3
81          add $t1 $t4 $zero
82          subi $sp $sp 3
83          sw $sp $t1 0
```

```
        addi $sp $sp 3
        j label2
label3
        subi $t5 $sp 2
        lw $t5 $t5 0
        addi $ret $t5 0
        subi $sp $sp 5
        jr
main
        addi $sp $sp 1
        subi $t6 $sp 0
        lw $t6 $t6 0
        in
        addi $t7 $RE 0
        add $t6 $t7 $zero
        subi $sp $sp 0
        sw $sp $t6 0
        addi $sp $sp 0
        subi $t8 $sp 0
        lw $t8 $t8 0
        addi $a0 $t8 0
        addi $sp $sp 1
        sw $sp $t8 0
        addi $sp $sp 1
        sw $sp $ra 0
        jal fibonacci
        lw $ra $sp 0
        subi $sp $sp 1
        lw $t8 $sp 0
        subi $sp $sp 1
        add $t9 $ret $zero
        addi $a0 $t9 0
        out $a0 $zero
        subi $sp $sp 1
        jr
*/
```

5.2.4 Código binário

```
/* C digo bin rio gerado automaticamente
mem_instrucao[0] = { 6'd2, 5'd26, 5'd25, -16'd1 }    //addi
mem_instrucao[1] = { 6'd2, 5'd26, 5'd27, 16'd0 }    //addi
mem_instrucao[2] = { 6'd27, 26'd91 }          //jal main
mem_instrucao[3] = { 6'd28, 26'd0 }   //break
mem_instrucao[4] = { 32'd0 }//fibonacci
mem_instrucao[5] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[6] = { 6'd9, 5'd17, 5'd25, 16'd0 }      //sw
mem_instrucao[7] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[8] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[9] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[10] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[11] = { 6'd4, 5'd25, 5'd0, 16'd3 }    //subi
mem_instrucao[12] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
mem_instrucao[13] = { 6'd2, 5'd26, 5'd1, 16'd0 }    //addi
mem_instrucao[14] = { 6'd1, 5'd1, 5'd26, 5'd0, 11'd0 }     //add
mem_instrucao[15] = { 6'd4, 5'd25, 5'd25, 16'd3 }    //subi
mem_instrucao[16] = { 6'd9, 5'd0, 5'd25, 16'd0 }      //sw
mem_instrucao[17] = { 6'd2, 5'd25, 5'd25, 16'd3 }    //addi
mem_instrucao[18] = { 32'd0 }  //label2
```

```
21 mem_instrucao[19] = { 6'd4, 5'd25, 5'd2, 16'd3 }    //subi
22 mem_instrucao[20] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
23 mem_instrucao[21] = { 6'd4, 5'd25, 5'd3, 16'd4 }    //subi
24 mem_instrucao[22] = { 6'd8, 5'd3, 5'd3, 16'd0 }      //lw
25 mem_instrucao[23] = { 6'd20, 5'd2, 5'd3, 5'd4, 11'd0 }     //cmp
26 mem_instrucao[24] = { 6'd2, 5'd26, 5'd2, 16'd2 }    //addi
27 mem_instrucao[25] = { 6'd19, 5'd4, 5'd2, 5'd4, 11'd0 }      //eq
28 mem_instrucao[26] = { 6'd22, 5'd4, 5'd0, 5'd4, 11'd0 }     //not
29 mem_instrucao[27] = { 6'd25, 5'd4, 5'd26, 16'd58 }   //beq label3
30 mem_instrucao[28] = { 6'd4, 5'd25, 5'd5, 16'd3 }    //subi
31 mem_instrucao[29] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
32 mem_instrucao[30] = { 6'd2, 5'd26, 5'd6, 16'd1 }    //addi
33 mem_instrucao[31] = { 6'd20, 5'd5, 5'd6, 5'd7, 11'd0 }     //cmp
34 mem_instrucao[32] = { 6'd2, 5'd26, 5'd5, 16'd2 }    //addi
35 mem_instrucao[33] = { 6'd19, 5'd7, 5'd5, 5'd7, 11'd0 }      //eq
36 mem_instrucao[34] = { 6'd22, 5'd7, 5'd0, 5'd7, 11'd0 }     //not
37 mem_instrucao[35] = { 6'd25, 5'd7, 5'd26, 16'd10 }   //beq label0
38 mem_instrucao[36] = { 6'd4, 5'd25, 5'd8, 16'd2 }    //subi
39 mem_instrucao[37] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
40 mem_instrucao[38] = { 6'd4, 5'd25, 5'd9, 16'd3 }    //subi
41 mem_instrucao[39] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
42 mem_instrucao[40] = { 6'd1, 5'd9, 5'd26, 5'd8, 11'd0 }     //add
43 mem_instrucao[41] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
44 mem_instrucao[42] = { 6'd9, 5'd8, 5'd25, 16'd0 }      //sw
45 mem_instrucao[43] = { 6'd2, 5'd25, 5'd25, 16'd2 }    //addi
46 mem_instrucao[44] = { 6'd23, 26'd73 }          //j label1
47 mem_instrucao[45] = { 32'd0 }  //label0
48 mem_instrucao[46] = { 6'd4, 5'd25, 5'd10, 16'd2 }    //subi
49 mem_instrucao[47] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
50 mem_instrucao[48] = { 6'd4, 5'd25, 5'd11, 16'd1 }    //subi
51 mem_instrucao[49] = { 6'd8, 5'd11, 5'd11, 16'd0 }      //lw
52 mem_instrucao[50] = { 6'd4, 5'd25, 5'd12, 16'd0 }    //subi
53 mem_instrucao[51] = { 6'd8, 5'd12, 5'd12, 16'd0 }      //lw
54 mem_instrucao[52] = { 6'd1, 5'd11, 5'd12, 5'd13, 11'd0 }     //add
55 mem_instrucao[53] = { 6'd1, 5'd13, 5'd26, 5'd10, 11'd0 }     //add
56 mem_instrucao[54] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
57 mem_instrucao[55] = { 6'd9, 5'd10, 5'd25, 16'd0 }      //sw
58 mem_instrucao[56] = { 6'd2, 5'd25, 5'd25, 16'd2 }    //addi
59 mem_instrucao[57] = { 6'd4, 5'd25, 5'd14, 16'd1 }    //subi
60 mem_instrucao[58] = { 6'd8, 5'd14, 5'd14, 16'd0 }      //lw
61 mem_instrucao[59] = { 6'd4, 5'd25, 5'd15, 16'd0 }    //subi
62 mem_instrucao[60] = { 6'd8, 5'd15, 5'd15, 16'd0 }      //lw
63 mem_instrucao[61] = { 6'd1, 5'd15, 5'd26, 5'd14, 11'd0 }     //add
64 mem_instrucao[62] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
65 mem_instrucao[63] = { 6'd9, 5'd14, 5'd25, 16'd0 }      //sw
66 mem_instrucao[64] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
67 mem_instrucao[65] = { 6'd4, 5'd25, 5'd16, 16'd0 }    //subi
68 mem_instrucao[66] = { 6'd8, 5'd16, 5'd16, 16'd0 }      //lw
69 mem_instrucao[67] = { 6'd4, 5'd25, 5'd0, 16'd2 }    //subi
70 mem_instrucao[68] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
71 mem_instrucao[69] = { 6'd1, 5'd0, 5'd26, 5'd16, 11'd0 }     //add
72 mem_instrucao[70] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
73 mem_instrucao[71] = { 6'd9, 5'd16, 5'd25, 16'd0 }      //sw
74 mem_instrucao[72] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
75 mem_instrucao[73] = { 32'd0 }  //label1
76 mem_instrucao[74] = { 6'd4, 5'd25, 5'd1, 16'd3 }    //subi
77 mem_instrucao[75] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
78 mem_instrucao[76] = { 6'd4, 5'd25, 5'd2, 16'd3 }    //subi
79 mem_instrucao[77] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
80 mem_instrucao[78] = { 6'd2, 5'd26, 5'd3, 16'd1 }    //addi
```

```
mem_instrucao[79] = { 6'd1, 5'd2, 5'd3, 5'd4, 11'd0 }     //add
mem_instrucao[80] = { 6'd1, 5'd4, 5'd26, 5'd1, 11'd0 }     //add
mem_instrucao[81] = { 6'd4, 5'd25, 5'd25, 16'd3 }    //subi
mem_instrucao[82] = { 6'd9, 5'd1, 5'd25, 16'd0 }      //sw
mem_instrucao[83] = { 6'd2, 5'd25, 5'd25, 16'd3 }    //addi
mem_instrucao[84] = { 6'd23, 26'd18 }          //j label2
mem_instrucao[85] = { 32'd0 }  //label3
mem_instrucao[86] = { 6'd4, 5'd25, 5'd5, 16'd2 }    //subi
mem_instrucao[87] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
mem_instrucao[88] = { 6'd2, 5'd5, 5'd24, 16'd0 }    //addi
mem_instrucao[89] = { 6'd4, 5'd25, 5'd25, 16'd5 }    //subi
mem_instrucao[90] = { 6'd24, 26'd0 }      //jr
mem_instrucao[91] = { 32'd0 }    //main
mem_instrucao[92] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[93] = { 6'd4, 5'd25, 5'd6, 16'd0 }    //subi
mem_instrucao[94] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
mem_instrucao[95] = { 6'd30, 26'd0 }      //in
mem_instrucao[96] = { 6'd2, 5'd28, 5'd7, 16'd0 }    //addi
mem_instrucao[97] = { 6'd1, 5'd7, 5'd26, 5'd6, 11'd0 }     //add
mem_instrucao[98] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[99] = { 6'd9, 5'd6, 5'd25, 16'd0 }      //sw
mem_instrucao[100] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[101] = { 6'd4, 5'd25, 5'd8, 16'd0 }    //subi
mem_instrucao[102] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
mem_instrucao[103] = { 6'd2, 5'd8, 5'd17, 16'd0 }    //addi
mem_instrucao[104] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[105] = { 6'd9, 5'd8, 5'd25, 16'd0 }      //sw
mem_instrucao[106] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[107] = { 6'd9, 5'd31, 5'd25, 16'd0 }      //sw
mem_instrucao[108] = { 6'd27, 26'd4 }     //jal fibonacci
mem_instrucao[109] = { 6'd8, 5'd25, 5'd31, 16'd0 }      //lw
mem_instrucao[110] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[111] = { 6'd8, 5'd25, 5'd8, 16'd0 }      //lw
mem_instrucao[112] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[113] = { 6'd1, 5'd24, 5'd26, 5'd9, 11'd0 }     //add
mem_instrucao[114] = { 6'd2, 5'd9, 5'd17, 16'd0 }    //addi
mem_instrucao[115] = { 6'd29, 5'd17, 5'd26, 16'd0 }     //out
mem_instrucao[116] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[117] = { 6'd24, 26'd0 }      //jr
*/
```

5.3 Exemplo 3: Sort

] Esse algoritmo ordena um vetor de entrada.

```


int vet[ 10 ];

int minloc ( int a[], int low, int high )
{       int i; int x; int k;
        k = low;
        x = a[low];
        i = low + 1;
        while (i < high){
                if (a[i] < x){
                        x = a[i];
                        k = i;
                }
```

```
15                  i = i + 1;
16          }
17          return k;
18  }
19
20  void sort( int a[], int low, int high)
21  {       int i; int k;
22          i = low;
23          while (i < high-1){
24                  int t;
25                  k = minloc(a,i,high);
26                  t = a[k];
27                  a[k] = a[i];
28                  a[i] = t;
29                  i = i + 1;
30          }
31  }
32
33  void main(void)
34  {
35          int i;
36          i = 0;
37          while (i < 10){
38                  vet[i] = input();
39                  i = i + 1;
40          }
41          sort(vet,0,10);
42          i = 0;
43          while (i < 10){
44                  output(vet[i]);
45                  i = i + 1;
46          }
47  }
```

5.3.1 Código intermediário

```
1   /*
2   <alloc: vet, global, 10>
3   <fun: int, minloc, -->
4   <param: vector, a, minloc>
5   <param: int, low, minloc>
6   <param: int, high, minloc>
7   <alloc: i, minloc, 1>
8   <alloc: x, minloc, 1>
9   <alloc: k, minloc, 1>
10  <load: $t0, k, -->
11  <load: $t1, low, -->
12  <assign: $t0, $t1, -->
13  <store: k, $t0, $zero>
14  <load: $t2, x, -->
15  <load: $t3, low, -->
16  <load: $t4, a, $t3>
17  <assign: $t2, $t4, -->
18  <store: x, $t2, $zero>
19  <load: $t5, i, -->
20  <load: $t6, low, -->
21  <immed: $t7, 1, -->
22  <add: $t8, $t6, $t7>
```

```
<assign: $t5, $t8, -->
<store: i, $t5, $zero>
<lab: label1, --, -->
<load: $t9, i, -->
<load: $t10, high, -->
<lt: $t11, $t9, $t10>
<iff: $t11, label2, -->
<load: $t12, i, -->
<load: $t13, a, $t12>
<load: $t14, x, -->
<lt: $t15, $t13, $t14>
<iff: $t15, label0, -->
<load: $t16, x, -->
<load: $t17, i, -->
<load: $t18, a, $t17>
<assign: $t16, $t18, -->
<store: x, $t16, $zero>
<load: $t19, k, -->
<load: $t20, i, -->
<assign: $t19, $t20, -->
<store: k, $t19, $zero>
<lab: label0, --, -->
<load: $t21, i, -->
<load: $t22, i, -->
<immed: $t23, 1, -->
<add: $t24, $t22, $t23>
<assign: $t21, $t24, -->
<store: i, $t21, $zero>
<goto: label1, --, -->
<lab: label2, --, -->
<load: $t25, k, -->
<ret: $t25, --, -->
<end: minloc, --, -->
<fun: void, sort, -->
<param: vector, a, sort>
<param: int, low, sort>
<param: int, high, sort>
<alloc: i, sort, 1>
<alloc: k, sort, 1>
<load: $t26, i, -->
<load: $t27, low, -->
<assign: $t26, $t27, -->
<store: i, $t26, $zero>
<lab: label3, --, -->
<load: $t28, i, -->
<load: $t29, high, -->
<immed: $t30, 1, -->
<sub: $t31, $t29, $t30>
<lt: $t32, $t28, $t31>
<iff: $t32, label4, -->
<alloc: t, sort, 1>
<load: $t33, k, -->
<load: $t34, a, -->
<arg: $t34, 1, 0>
<load: $t35, i, -->
<arg: $t35, --, -->
<load: $t36, high, -->
<arg: $t36, --, -->
<call: $t37, minloc, 3>
<assign: $t33, $t37, -->
```

```
83  <store: k, $t33, $zero>
84  <load: $t38, t, -->
85  <load: $t39, k, -->
86  <load: $t40, a, $t39>
87  <assign: $t38, $t40, -->
88  <store: t, $t38, $zero>
89  <load: $t41, k, -->
90  <load: $t42, a, $t41>
91  <load: $t43, i, -->
92  <load: $t44, a, $t43>
93  <assign: $t42, $t44, -->
94  <store: a, $t42, $t41>
95  <load: $t45, i, -->
96  <load: $t46, a, $t45>
97  <load: $t47, t, -->
98  <assign: $t46, $t47, -->
99  <store: a, $t46, $t45>
100 <load: $t48, i, -->
101 <load: $t49, i, -->
102 <immed: $t50, 1, -->
103 <add: $t51, $t49, $t50>
104 <assign: $t48, $t51, -->
105 <store: i, $t48, $zero>
106 <goto: label3, --, -->
107 <lab: label4, --, -->
108 <end: sort, --, -->
109 <fun: void, main, -->
110 <alloc: i, main, 1>
111 <load: $t52, i, -->
112 <immed: $t53, 0, -->
113 <assign: $t52, $t53, -->
114 <store: i, $t52, $zero>
115 <lab: label5, --, -->
116 <load: $t54, i, -->
117 <immed: $t55, 10, -->
118 <lt: $t56, $t54, $t55>
119 <iff: $t56, label6, -->
120 <load: $t57, i, -->
121 <load: $t58, vet, $t57>
122 <input: $t59, input, 0>
123 <assign: $t58, $t59, -->
124 <store: vet, $t58, $t57>
125 <load: $t60, i, -->
126 <load: $t61, i, -->
127 <immed: $t62, 1, -->
128 <add: $t63, $t61, $t62>
129 <assign: $t60, $t63, -->
130 <store: i, $t60, $zero>
131 <goto: label5, --, -->
132 <lab: label6, --, -->
133 <load: $t64, vet, -->
134 <arg: $t64, 0, 0>
135 <immed: $t65, 0, -->
136 <arg: $t65, --, -->
137 <immed: $t66, 10, -->
138 <arg: $t66, --, -->
139 <call: $t67, sort, 3>
140 <load: $t68, i, -->
141 <immed: $t69, 0, -->
142 <assign: $t68, $t69, -->
```

```
<store: i, $t68, $zero>
<lab: label7, --, -->
<load: $t70, i, -->
<immed: $t71, 10, -->
<lt: $t72, $t70, $t71>
<iff: $t72, label8, -->
<load: $t73, i, -->
<load: $t74, vet, $t73>
<arg: $t74, --, -->
<output: $t75, output, 1>
<load: $t76, i, -->
<load: $t77, i, -->
<immed: $t78, 1, -->
<add: $t79, $t77, $t78>
<assign: $t76, $t79, -->
<store: i, $t76, $zero>
<goto: label7, --, -->
<lab: label8, --, -->
<end: main, --, -->
*/
```

5.3.2 Código assembly

```
/*      addi $sp $zero 9
        addi $gp $zero 0
        jal main
        break
minloc
        addi $sp $sp 1
        sw $sp $a0 0
        addi $sp $sp 1
        sw $sp $a1 0
        addi $sp $sp 1
        sw $sp $a2 0
        addi $sp $sp 1
        addi $sp $sp 1
        addi $sp $sp 1
        subi $t0 $sp 0
        lw $t0 $t0 0
        subi $t1 $sp 4
        lw $t1 $t1 0
        add $t0 $t1 $zero
        subi $sp $sp 0
        sw $sp $t0 0
        addi $sp $sp 0
        subi $t2 $sp 1
        lw $t2 $t2 0
        subi $t3 $sp 4
        lw $t3 $t3 0
        subi $t4 $sp 5
        lw $t4 $t4 0
        add $t4 $t4 $t3
        lw $t4 $t4 0
        add $t2 $t4 $zero
        subi $sp $sp 1
        sw $sp $t2 0
        addi $sp $sp 1
        subi $t5 $sp 2
        lw $t5 $t5 0
```

```
37          subi $t6 $sp 4
38          lw $t6 $t6 0
39          addi $t7 $zero 1
40          add $t8 $t6 $t7
41          add $t5 $t8 $zero
42          subi $sp $sp 2
43          sw $sp $t5 0
44          addi $sp $sp 2
45  label1
46          subi $t9 $sp 2
47          lw $t9 $t9 0
48          subi $t10 $sp 3
49          lw $t10 $t10 0
50          cmp $t11 $t9 $t10
51          eq $t11 $t11 $zero
52          beq $t11 $zero label2
53          subi $t12 $sp 2
54          lw $t12 $t12 0
55          subi $t13 $sp 5
56          lw $t13 $t13 0
57          add $t13 $t13 $t12
58          lw $t13 $t13 0
59          subi $t14 $sp 1
60          lw $t14 $t14 0
61          cmp $t15 $t13 $t14
62          eq $t15 $t15 $zero
63          beq $t15 $zero label0
64          subi $t16 $sp 1
65          lw $t16 $t16 0
66          subi $t0 $sp 2
67          lw $t0 $t0 0
68          subi $t1 $sp 5
69          lw $t1 $t1 0
70          add $t1 $t1 $t0
71          lw $t1 $t1 0
72          add $t16 $t1 $zero
73          subi $sp $sp 1
74          sw $sp $t16 0
75          addi $sp $sp 1
76          subi $t2 $sp 0
77          lw $t2 $t2 0
78          subi $t3 $sp 2
79          lw $t3 $t3 0
80          add $t2 $t3 $zero
81          subi $sp $sp 0
82          sw $sp $t2 0
83          addi $sp $sp 0
84  label0
85          subi $t4 $sp 2
86          lw $t4 $t4 0
87          subi $t5 $sp 2
88          lw $t5 $t5 0
89          addi $t6 $zero 1
90          add $t7 $t5 $t6
91          add $t4 $t7 $zero
92          subi $sp $sp 2
93          sw $sp $t4 0
94          addi $sp $sp 2
95          j label1
96  label2
```

```
        subi $t8 $sp 0
        lw $t8 $t8 0
        addi $ret $t8 0
        subi $sp $sp 6
        jr
sort
        addi $sp $sp 1
        sw $sp $a0 0
        addi $sp $sp 1
        sw $sp $a1 0
        addi $sp $sp 1
        sw $sp $a2 0
        addi $sp $sp 1
        addi $sp $sp 1
        subi $t9 $sp 1
        lw $t9 $t9 0
        subi $t10 $sp 3
        lw $t10 $t10 0
        add $t9 $t10 $zero
        subi $sp $sp 1
        sw $sp $t9 0
        addi $sp $sp 1
label3
        subi $t11 $sp 1
        lw $t11 $t11 0
        subi $t12 $sp 2
        lw $t12 $t12 0
        addi $t13 $zero 1
        sub $t14 $t12 $t13
        cmp $t15 $t11 $t14
        eq $t15 $t15 $zero
        beq $t15 $zero label4
        addi $sp $sp 1
        subi $t16 $sp 1
        lw $t16 $t16 0
        subi $t0 $sp 5
        lw $t0 $t0 0
        addi $a0 $t0 0
        subi $t1 $sp 2
        lw $t1 $t1 0
        addi $a1 $t1 0
        subi $t2 $sp 3
        lw $t2 $t2 0
        addi $a2 $t2 0
        addi $sp $sp 1
        sw $sp $t0 0
        addi $sp $sp 1
        sw $sp $t1 0
        addi $sp $sp 1
        sw $sp $t2 0
        addi $sp $sp 1
        sw $sp $ra 0
        jal minloc
        lw $ra $sp 0
        subi $sp $sp 1
        lw $t2 $sp 0
        subi $sp $sp 1
        lw $t1 $sp 0
        subi $sp $sp 1
        lw $t0 $sp 0
```

```
157         subi $sp $sp 1
158         add $t3 $ret $zero
159         add $t16 $t3 $zero
160         subi $sp $sp 1
161         sw $sp $t16 0
162         addi $sp $sp 1
163         subi $t4 $sp 0
164         lw $t4 $t4 0
165         subi $t5 $sp 1
166         lw $t5 $t5 0
167         subi $t6 $sp 5
168         lw $t6 $t6 0
169         add $t6 $t6 $t5
170         lw $t6 $t6 0
171         add $t4 $t6 $zero
172         subi $sp $sp 0
173         sw $sp $t4 0
174         addi $sp $sp 0
175         subi $t7 $sp 1
176         lw $t7 $t7 0
177         subi $t8 $sp 5
178         lw $t8 $t8 0
179         add $t8 $t8 $t7
180         lw $t8 $t8 0
181         subi $t9 $sp 2
182         lw $t9 $t9 0
183         subi $t10 $sp 5
184         lw $t10 $t10 0
185         add $t10 $t10 $t9
186         lw $t10 $t10 0
187         add $t8 $t10 $zero
188         subi $t9 $sp 5
189         lw $t9 $t9 0
190         add $t9 $t9 $t7
191         sw $t9 $t8 0
192         subi $t11 $sp 2
193         lw $t11 $t11 0
194         subi $t12 $sp 5
195         lw $t12 $t12 0
196         add $t12 $t12 $t11
197         lw $t12 $t12 0
198         subi $t13 $sp 0
199         lw $t13 $t13 0
200         add $t12 $t13 $zero
201         subi $t13 $sp 5
202         lw $t13 $t13 0
203         add $t13 $t13 $t11
204         sw $t13 $t12 0
205         subi $t14 $sp 2
206         lw $t14 $t14 0
207         subi $t15 $sp 2
208         lw $t15 $t15 0
209         addi $t16 $zero 1
210         add $t0 $t15 $t16
211         add $t14 $t0 $zero
212         subi $sp $sp 2
213         sw $sp $t14 0
214         addi $sp $sp 2
215         j label3
216 label4
```

```
        subi $sp $sp 6
        jr
main
        addi $sp $sp 1
        subi $t1 $sp 0
        lw $t1 $t1 0
        addi $t2 $zero 0
        add $t1 $t2 $zero
        subi $sp $sp 0
        sw $sp $t1 0
        addi $sp $sp 0
label5
        subi $t3 $sp 0
        lw $t3 $t3 0
        addi $t4 $zero 10
        cmp $t5 $t3 $t4
        eq $t5 $t5 $zero
        beq $t5 $zero label6
        subi $t6 $sp 0
        lw $t6 $t6 0
        add $t7 $gp $t6
        lw $t7 $t7 0
        in
        addi $t8 $RE 0
        add $t7 $t8 $zero
        add $t6 $gp $t6
        sw $t6 $gp 0
        subi $t9 $sp 0
        lw $t9 $t9 0
        subi $t10 $sp 0
        lw $t10 $t10 0
        addi $t11 $zero 1
        add $t12 $t10 $t11
        add $t9 $t12 $zero
        subi $sp $sp 0
        sw $sp $t9 0
        addi $sp $sp 0
        j label5
label6
        addi $a0 $zero 0
        addi $t14 $zero 0
        addi $a1 $t14 0
        addi $t15 $zero 10
        addi $a2 $t15 0
        addi $sp $sp 1
        sw $sp $t6 0
        addi $sp $sp 1
        sw $sp $t14 0
        addi $sp $sp 1
        sw $sp $ra 0
        jal sort
        lw $ra $sp 0
        subi $sp $sp 1
        lw $t14 $sp 0
        subi $sp $sp 1
        lw $t6 $sp 0
        subi $sp $sp 1
        subi $t0 $sp 0
        lw $t0 $t0 0
        addi $t1 $zero 0
```

```
        add $t0 $t1 $zero
        subi $sp $sp 0
        sw $sp $t0 0
        addi $sp $sp 0
label7
        subi $t2 $sp 0
        lw $t2 $t2 0
        addi $t3 $zero 10
        cmp $t4 $t2 $t3
        eq $t4 $t4 $zero
        beq $t4 $zero label8
        subi $t5 $sp 0
        lw $t5 $t5 0
        add $t6 $gp $t5
        lw $t6 $t6 0
        addi $a0 $t6 0
        out $a0 $zero
        subi $t8 $sp 0
        lw $t8 $t8 0
        subi $t9 $sp 0
        lw $t9 $t9 0
        addi $t10 $zero 1
        add $t11 $t9 $t10
        add $t8 $t11 $zero
        subi $sp $sp 0
        sw $sp $t8 0
        addi $sp $sp 0
        j label7
label8
        subi $sp $sp 1
        jr
*/
```

5.3.3 Código binário

```
/* C digo bin rio gerado automaticamente
mem_instrucao[0] = { 6'd2, 5'd26, 5'd25, 16'd9 }    //addi
mem_instrucao[1] = { 6'd2, 5'd26, 5'd27, 16'd0 }    //addi
mem_instrucao[2] = { 6'd27, 26'd218 }          //jal main
mem_instrucao[3] = { 6'd28, 26'd0 }   //break
mem_instrucao[4] = { 32'd0 }  //minloc
mem_instrucao[5] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[6] = { 6'd9, 5'd17, 5'd25, 16'd0 }      //sw
mem_instrucao[7] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[8] = { 6'd9, 5'd18, 5'd25, 16'd0 }      //sw
mem_instrucao[9] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[10] = { 6'd9, 5'd19, 5'd25, 16'd0 }      //sw
mem_instrucao[11] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[12] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[13] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[14] = { 6'd4, 5'd25, 5'd0, 16'd0 }    //subi
mem_instrucao[15] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
mem_instrucao[16] = { 6'd4, 5'd25, 5'd1, 16'd4 }    //subi
mem_instrucao[17] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
mem_instrucao[18] = { 6'd1, 5'd1, 5'd26, 5'd0, 11'd0 }     //add
mem_instrucao[19] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[20] = { 6'd9, 5'd0, 5'd25, 16'd0 }      //sw
mem_instrucao[21] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[22] = { 6'd4, 5'd25, 5'd2, 16'd1 }    //subi
```

```
25 mem_instrucao[23] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
26 mem_instrucao[24] = { 6'd4, 5'd25, 5'd3, 16'd4 }    //subi
27 mem_instrucao[25] = { 6'd8, 5'd3, 5'd3, 16'd0 }      //lw
28 mem_instrucao[26] = { 6'd4, 5'd25, 5'd4, 16'd5 }    //subi
29 mem_instrucao[27] = { 6'd8, 5'd4, 5'd4, 16'd0 }      //lw
30 mem_instrucao[28] = { 6'd1, 5'd4, 5'd3, 5'd4, 11'd0 }     //add
31 mem_instrucao[29] = { 6'd8, 5'd4, 5'd4, 16'd0 }      //lw
32 mem_instrucao[30] = { 6'd1, 5'd4, 5'd26, 5'd2, 11'd0 }     //add
33 mem_instrucao[31] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
34 mem_instrucao[32] = { 6'd9, 5'd2, 5'd25, 16'd0 }      //sw
35 mem_instrucao[33] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
36 mem_instrucao[34] = { 6'd4, 5'd25, 5'd5, 16'd2 }    //subi
37 mem_instrucao[35] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
38 mem_instrucao[36] = { 6'd4, 5'd25, 5'd6, 16'd4 }    //subi
39 mem_instrucao[37] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
40 mem_instrucao[38] = { 6'd2, 5'd26, 5'd7, 16'd1 }    //addi
41 mem_instrucao[39] = { 6'd1, 5'd6, 5'd7, 5'd8, 11'd0 }     //add
42 mem_instrucao[40] = { 6'd1, 5'd8, 5'd26, 5'd5, 11'd0 }     //add
43 mem_instrucao[41] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
44 mem_instrucao[42] = { 6'd9, 5'd5, 5'd25, 16'd0 }      //sw
45 mem_instrucao[43] = { 6'd2, 5'd25, 5'd25, 16'd2 }    //addi
46 mem_instrucao[44] = { 32'd0 }  //label1
47 mem_instrucao[45] = { 6'd4, 5'd25, 5'd9, 16'd2 }    //subi
48 mem_instrucao[46] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
49 mem_instrucao[47] = { 6'd4, 5'd25, 5'd10, 16'd3 }    //subi
50 mem_instrucao[48] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
51 mem_instrucao[49] = { 6'd20, 5'd9, 5'd10, 5'd11, 11'd0 }     //cmp
52 mem_instrucao[50] = { 6'd19, 5'd11, 5'd26, 5'd11, 11'd0 }      //eq
53 mem_instrucao[51] = { 6'd25, 5'd11, 5'd26, 16'd44 }   //beq label2
54 mem_instrucao[52] = { 6'd4, 5'd25, 5'd12, 16'd2 }    //subi
55 mem_instrucao[53] = { 6'd8, 5'd12, 5'd12, 16'd0 }      //lw
56 mem_instrucao[54] = { 6'd4, 5'd25, 5'd13, 16'd5 }    //subi
57 mem_instrucao[55] = { 6'd8, 5'd13, 5'd13, 16'd0 }      //lw
58 mem_instrucao[56] = { 6'd1, 5'd13, 5'd12, 5'd13, 11'd0 }     //add
59 mem_instrucao[57] = { 6'd8, 5'd13, 5'd13, 16'd0 }      //lw
60 mem_instrucao[58] = { 6'd4, 5'd25, 5'd14, 16'd1 }    //subi
61 mem_instrucao[59] = { 6'd8, 5'd14, 5'd14, 16'd0 }      //lw
62 mem_instrucao[60] = { 6'd20, 5'd13, 5'd14, 5'd15, 11'd0 }     //cmp
63 mem_instrucao[61] = { 6'd19, 5'd15, 5'd26, 5'd15, 11'd0 }      //eq
64 mem_instrucao[62] = { 6'd25, 5'd15, 5'd26, 16'd21 }   //beq label0
65 mem_instrucao[63] = { 6'd4, 5'd25, 5'd16, 16'd1 }    //subi
66 mem_instrucao[64] = { 6'd8, 5'd16, 5'd16, 16'd0 }      //lw
67 mem_instrucao[65] = { 6'd4, 5'd25, 5'd0, 16'd2 }    //subi
68 mem_instrucao[66] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
69 mem_instrucao[67] = { 6'd4, 5'd25, 5'd1, 16'd5 }    //subi
70 mem_instrucao[68] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
71 mem_instrucao[69] = { 6'd1, 5'd1, 5'd0, 5'd1, 11'd0 }     //add
72 mem_instrucao[70] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
73 mem_instrucao[71] = { 6'd1, 5'd1, 5'd26, 5'd16, 11'd0 }     //add
74 mem_instrucao[72] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
75 mem_instrucao[73] = { 6'd9, 5'd16, 5'd25, 16'd0 }      //sw
76 mem_instrucao[74] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
77 mem_instrucao[75] = { 6'd4, 5'd25, 5'd2, 16'd0 }    //subi
78 mem_instrucao[76] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
79 mem_instrucao[77] = { 6'd4, 5'd25, 5'd3, 16'd2 }    //subi
80 mem_instrucao[78] = { 6'd8, 5'd3, 5'd3, 16'd0 }      //lw
81 mem_instrucao[79] = { 6'd1, 5'd3, 5'd26, 5'd2, 11'd0 }     //add
82 mem_instrucao[80] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
83 mem_instrucao[81] = { 6'd9, 5'd2, 5'd25, 16'd0 }      //sw
84 mem_instrucao[82] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
```

```
85  mem_instrucao[83] = { 32'd0 }  //label0
86  mem_instrucao[84] = { 6'd4, 5'd25, 5'd4, 16'd2 }    //subi
87  mem_instrucao[85] = { 6'd8, 5'd4, 5'd4, 16'd0 }      //lw
88  mem_instrucao[86] = { 6'd4, 5'd25, 5'd5, 16'd2 }    //subi
89  mem_instrucao[87] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
90  mem_instrucao[88] = { 6'd2, 5'd26, 5'd6, 16'd1 }    //addi
91  mem_instrucao[89] = { 6'd1, 5'd5, 5'd6, 5'd7, 11'd0 }     //add
92  mem_instrucao[90] = { 6'd1, 5'd7, 5'd26, 5'd4, 11'd0 }     //add
93  mem_instrucao[91] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
94  mem_instrucao[92] = { 6'd9, 5'd4, 5'd25, 16'd0 }      //sw
95  mem_instrucao[93] = { 6'd2, 5'd25, 5'd25, 16'd2 }    //addi
96  mem_instrucao[94] = { 6'd23, 26'd44 }          //j label1
97  mem_instrucao[95] = { 32'd0 }  //label2
98  mem_instrucao[96] = { 6'd4, 5'd25, 5'd8, 16'd0 }    //subi
99  mem_instrucao[97] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
100 mem_instrucao[98] = { 6'd2, 5'd8, 5'd24, 16'd0 }    //addi
101 mem_instrucao[99] = { 6'd4, 5'd25, 5'd25, 16'd6 }    //subi
102 mem_instrucao[100] = { 6'd24, 26'd0 }      //jr
103 mem_instrucao[101] = { 32'd0 }    //sort
104 mem_instrucao[102] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
105 mem_instrucao[103] = { 6'd9, 5'd17, 5'd25, 16'd0 }      //sw
106 mem_instrucao[104] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
107 mem_instrucao[105] = { 6'd9, 5'd18, 5'd25, 16'd0 }      //sw
108 mem_instrucao[106] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
109 mem_instrucao[107] = { 6'd9, 5'd19, 5'd25, 16'd0 }      //sw
110 mem_instrucao[108] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
111 mem_instrucao[109] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
112 mem_instrucao[110] = { 6'd4, 5'd25, 5'd9, 16'd1 }    //subi
113 mem_instrucao[111] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
114 mem_instrucao[112] = { 6'd4, 5'd25, 5'd10, 16'd3 }    //subi
115 mem_instrucao[113] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
116 mem_instrucao[114] = { 6'd1, 5'd10, 5'd26, 5'd9, 11'd0 }     //add
117 mem_instrucao[115] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
118 mem_instrucao[116] = { 6'd9, 5'd9, 5'd25, 16'd0 }      //sw
119 mem_instrucao[117] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
120 mem_instrucao[118] = { 32'd0 }  //label3
121 mem_instrucao[119] = { 6'd4, 5'd25, 5'd11, 16'd1 }    //subi
122 mem_instrucao[120] = { 6'd8, 5'd11, 5'd11, 16'd0 }      //lw
123 mem_instrucao[121] = { 6'd4, 5'd25, 5'd12, 16'd2 }    //subi
124 mem_instrucao[122] = { 6'd8, 5'd12, 5'd12, 16'd0 }      //lw
125 mem_instrucao[123] = { 6'd2, 5'd26, 5'd13, 16'd1 }    //addi
126 mem_instrucao[124] = { 6'd3, 5'd12, 5'd13, 5'd14, 11'd0 }     //sub
127 mem_instrucao[125] = { 6'd20, 5'd11, 5'd14, 5'd15, 11'd0 }     //cmp
128 mem_instrucao[126] = { 6'd19, 5'd15, 5'd26, 5'd15, 11'd0 }      //eq
129 mem_instrucao[127] = { 6'd25, 5'd15, 5'd26, 16'd88 }   //beq label4
130 mem_instrucao[128] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
131 mem_instrucao[129] = { 6'd4, 5'd25, 5'd16, 16'd1 }    //subi
132 mem_instrucao[130] = { 6'd8, 5'd16, 5'd16, 16'd0 }      //lw
133 mem_instrucao[131] = { 6'd4, 5'd25, 5'd0, 16'd5 }    //subi
134 mem_instrucao[132] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
135 mem_instrucao[133] = { 6'd2, 5'd0, 5'd17, 16'd0 }    //addi
136 mem_instrucao[134] = { 6'd4, 5'd25, 5'd1, 16'd2 }    //subi
137 mem_instrucao[135] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
138 mem_instrucao[136] = { 6'd2, 5'd1, 5'd18, 16'd0 }    //addi
139 mem_instrucao[137] = { 6'd4, 5'd25, 5'd2, 16'd3 }    //subi
140 mem_instrucao[138] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
141 mem_instrucao[139] = { 6'd2, 5'd2, 5'd19, 16'd0 }    //addi
142 mem_instrucao[140] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
143 mem_instrucao[141] = { 6'd9, 5'd0, 5'd25, 16'd0 }      //sw
144 mem_instrucao[142] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
```

```
145 mem_instrucao[143] = { 6'd9, 5'd1, 5'd25, 16'd0 }      //sw
146 mem_instrucao[144] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
147 mem_instrucao[145] = { 6'd9, 5'd2, 5'd25, 16'd0 }      //sw
148 mem_instrucao[146] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
149 mem_instrucao[147] = { 6'd9, 5'd31, 5'd25, 16'd0 }      //sw
150 mem_instrucao[148] = { 6'd27, 26'd4 }        //jal minloc
151 mem_instrucao[149] = { 6'd8, 5'd25, 5'd31, 16'd0 }      //lw
152 mem_instrucao[150] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
153 mem_instrucao[151] = { 6'd8, 5'd25, 5'd2, 16'd0 }      //lw
154 mem_instrucao[152] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
155 mem_instrucao[153] = { 6'd8, 5'd25, 5'd1, 16'd0 }      //lw
156 mem_instrucao[154] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
157 mem_instrucao[155] = { 6'd8, 5'd25, 5'd0, 16'd0 }      //lw
158 mem_instrucao[156] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
159 mem_instrucao[157] = { 6'd1, 5'd24, 5'd26, 5'd3, 11'd0 }     //add
160 mem_instrucao[158] = { 6'd1, 5'd3, 5'd26, 5'd16, 11'd0 }     //add
161 mem_instrucao[159] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
162 mem_instrucao[160] = { 6'd9, 5'd16, 5'd25, 16'd0 }      //sw
163 mem_instrucao[161] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
164 mem_instrucao[162] = { 6'd4, 5'd25, 5'd4, 16'd0 }    //subi
165 mem_instrucao[163] = { 6'd8, 5'd4, 5'd4, 16'd0 }      //lw
166 mem_instrucao[164] = { 6'd4, 5'd25, 5'd5, 16'd1 }    //subi
167 mem_instrucao[165] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
168 mem_instrucao[166] = { 6'd4, 5'd25, 5'd6, 16'd5 }    //subi
169 mem_instrucao[167] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
170 mem_instrucao[168] = { 6'd1, 5'd6, 5'd5, 5'd6, 11'd0 }     //add
171 mem_instrucao[169] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
172 mem_instrucao[170] = { 6'd1, 5'd6, 5'd26, 5'd4, 11'd0 }     //add
173 mem_instrucao[171] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
174 mem_instrucao[172] = { 6'd9, 5'd4, 5'd25, 16'd0 }      //sw
175 mem_instrucao[173] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
176 mem_instrucao[174] = { 6'd4, 5'd25, 5'd7, 16'd1 }    //subi
177 mem_instrucao[175] = { 6'd8, 5'd7, 5'd7, 16'd0 }      //lw
178 mem_instrucao[176] = { 6'd4, 5'd25, 5'd8, 16'd5 }    //subi
179 mem_instrucao[177] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
180 mem_instrucao[178] = { 6'd1, 5'd8, 5'd7, 5'd8, 11'd0 }     //add
181 mem_instrucao[179] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
182 mem_instrucao[180] = { 6'd4, 5'd25, 5'd9, 16'd2 }    //subi
183 mem_instrucao[181] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
184 mem_instrucao[182] = { 6'd4, 5'd25, 5'd10, 16'd5 }    //subi
185 mem_instrucao[183] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
186 mem_instrucao[184] = { 6'd1, 5'd10, 5'd9, 5'd10, 11'd0 }     //add
187 mem_instrucao[185] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
188 mem_instrucao[186] = { 6'd1, 5'd10, 5'd26, 5'd8, 11'd0 }     //add
189 mem_instrucao[187] = { 6'd4, 5'd25, 5'd9, 16'd5 }    //subi
190 mem_instrucao[188] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
191 mem_instrucao[189] = { 6'd1, 5'd9, 5'd7, 5'd9, 11'd0 }     //add
192 mem_instrucao[190] = { 6'd9, 5'd8, 5'd9, 16'd0 }      //sw
193 mem_instrucao[191] = { 6'd4, 5'd25, 5'd11, 16'd2 }    //subi
194 mem_instrucao[192] = { 6'd8, 5'd11, 5'd11, 16'd0 }      //lw
195 mem_instrucao[193] = { 6'd4, 5'd25, 5'd12, 16'd5 }    //subi
196 mem_instrucao[194] = { 6'd8, 5'd12, 5'd12, 16'd0 }      //lw
197 mem_instrucao[195] = { 6'd1, 5'd12, 5'd11, 5'd12, 11'd0 }     //add
198 mem_instrucao[196] = { 6'd8, 5'd12, 5'd12, 16'd0 }      //lw
199 mem_instrucao[197] = { 6'd4, 5'd25, 5'd13, 16'd0 }    //subi
200 mem_instrucao[198] = { 6'd8, 5'd13, 5'd13, 16'd0 }      //lw
201 mem_instrucao[199] = { 6'd1, 5'd13, 5'd26, 5'd12, 11'd0 }     //add
202 mem_instrucao[200] = { 6'd4, 5'd25, 5'd13, 16'd5 }    //subi
203 mem_instrucao[201] = { 6'd8, 5'd13, 5'd13, 16'd0 }      //lw
204 mem_instrucao[202] = { 6'd1, 5'd13, 5'd11, 5'd13, 11'd0 }     //add
```

```
mem_instrucao[203] = { 6'd9, 5'd12, 5'd13, 16'd0 }      //sw
mem_instrucao[204] = { 6'd4, 5'd25, 5'd14, 16'd2 }    //subi
mem_instrucao[205] = { 6'd8, 5'd14, 5'd14, 16'd0 }      //lw
mem_instrucao[206] = { 6'd4, 5'd25, 5'd15, 16'd2 }    //subi
mem_instrucao[207] = { 6'd8, 5'd15, 5'd15, 16'd0 }      //lw
mem_instrucao[208] = { 6'd2, 5'd26, 5'd16, 16'd1 }    //addi
mem_instrucao[209] = { 6'd1, 5'd15, 5'd16, 5'd0, 11'd0 }     //add
mem_instrucao[210] = { 6'd1, 5'd0, 5'd26, 5'd14, 11'd0 }     //add
mem_instrucao[211] = { 6'd4, 5'd25, 5'd25, 16'd2 }    //subi
mem_instrucao[212] = { 6'd9, 5'd14, 5'd25, 16'd0 }      //sw
mem_instrucao[213] = { 6'd2, 5'd25, 5'd25, 16'd2 }    //addi
mem_instrucao[214] = { 6'd23, 26'd118 }          //j label3
mem_instrucao[215] = { 32'd0 }  //label4
mem_instrucao[216] = { 6'd4, 5'd25, 5'd25, 16'd6 }    //subi
mem_instrucao[217] = { 6'd24, 26'd0 }      //jr
mem_instrucao[218] = { 32'd0 }    //main
mem_instrucao[219] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[220] = { 6'd4, 5'd25, 5'd1, 16'd0 }    //subi
mem_instrucao[221] = { 6'd8, 5'd1, 5'd1, 16'd0 }      //lw
mem_instrucao[222] = { 6'd2, 5'd26, 5'd2, 16'd0 }    //addi
mem_instrucao[223] = { 6'd1, 5'd2, 5'd26, 5'd1, 11'd0 }     //add
mem_instrucao[224] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[225] = { 6'd9, 5'd1, 5'd25, 16'd0 }      //sw
mem_instrucao[226] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[227] = { 32'd0 }  //label5
mem_instrucao[228] = { 6'd4, 5'd25, 5'd3, 16'd0 }    //subi
mem_instrucao[229] = { 6'd8, 5'd3, 5'd3, 16'd0 }      //lw
mem_instrucao[230] = { 6'd2, 5'd26, 5'd4, 16'd10 }    //addi
mem_instrucao[231] = { 6'd20, 5'd3, 5'd4, 5'd5, 11'd0 }     //cmp
mem_instrucao[232] = { 6'd19, 5'd5, 5'd26, 5'd5, 11'd0 }      //eq
mem_instrucao[233] = { 6'd25, 5'd5, 5'd26, 16'd21 }   //beq label6
mem_instrucao[234] = { 6'd4, 5'd25, 5'd6, 16'd0 }    //subi
mem_instrucao[235] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
mem_instrucao[236] = { 6'd1, 5'd27, 5'd6, 5'd7, 11'd0 }     //add
mem_instrucao[237] = { 6'd8, 5'd7, 5'd7, 16'd0 }      //lw
mem_instrucao[238] = { 6'd30, 26'd0 }      //in
mem_instrucao[239] = { 6'd2, 5'd28, 5'd8, 16'd0 }    //addi
mem_instrucao[240] = { 6'd1, 5'd8, 5'd26, 5'd7, 11'd0 }     //add
mem_instrucao[241] = { 6'd1, 5'd27, 5'd6, 5'd6, 11'd0 }     //add
mem_instrucao[242] = { 6'd9, 5'd27, 5'd6, 16'd0 }      //sw
mem_instrucao[243] = { 6'd4, 5'd25, 5'd9, 16'd0 }    //subi
mem_instrucao[244] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
mem_instrucao[245] = { 6'd4, 5'd25, 5'd10, 16'd0 }    //subi
mem_instrucao[246] = { 6'd8, 5'd10, 5'd10, 16'd0 }      //lw
mem_instrucao[247] = { 6'd2, 5'd26, 5'd11, 16'd1 }    //addi
mem_instrucao[248] = { 6'd1, 5'd10, 5'd11, 5'd12, 11'd0 }     //add
mem_instrucao[249] = { 6'd1, 5'd12, 5'd26, 5'd9, 11'd0 }     //add
mem_instrucao[250] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[251] = { 6'd9, 5'd9, 5'd25, 16'd0 }      //sw
mem_instrucao[252] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[253] = { 6'd23, 26'd227 }          //j label5
mem_instrucao[254] = { 32'd0 }  //label6
mem_instrucao[255] = { 6'd2, 5'd26, 5'd17, 16'd0 }    //addi
mem_instrucao[256] = { 6'd2, 5'd26, 5'd14, 16'd0 }    //addi
mem_instrucao[257] = { 6'd2, 5'd14, 5'd18, 16'd0 }    //addi
mem_instrucao[258] = { 6'd2, 5'd26, 5'd15, 16'd10 }    //addi
mem_instrucao[259] = { 6'd2, 5'd15, 5'd19, 16'd0 }    //addi
mem_instrucao[260] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[261] = { 6'd9, 5'd6, 5'd25, 16'd0 }      //sw
mem_instrucao[262] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
```

```
mem_instrucao[263] = { 6'd9, 5'd14, 5'd25, 16'd0 }      //sw
mem_instrucao[264] = { 6'd2, 5'd25, 5'd25, 16'd1 }    //addi
mem_instrucao[265] = { 6'd9, 5'd31, 5'd25, 16'd0 }      //sw
mem_instrucao[266] = { 6'd27, 26'd101 }          //jal sort
mem_instrucao[267] = { 6'd8, 5'd25, 5'd31, 16'd0 }      //lw
mem_instrucao[268] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[269] = { 6'd8, 5'd25, 5'd14, 16'd0 }      //lw
mem_instrucao[270] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[271] = { 6'd8, 5'd25, 5'd6, 16'd0 }      //lw
mem_instrucao[272] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[273] = { 6'd4, 5'd25, 5'd0, 16'd0 }    //subi
mem_instrucao[274] = { 6'd8, 5'd0, 5'd0, 16'd0 }      //lw
mem_instrucao[275] = { 6'd2, 5'd26, 5'd1, 16'd0 }    //addi
mem_instrucao[276] = { 6'd1, 5'd1, 5'd26, 5'd0, 11'd0 }     //add
mem_instrucao[277] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[278] = { 6'd9, 5'd0, 5'd25, 16'd0 }      //sw
mem_instrucao[279] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[280] = { 32'd0 }  //label7
mem_instrucao[281] = { 6'd4, 5'd25, 5'd2, 16'd0 }    //subi
mem_instrucao[282] = { 6'd8, 5'd2, 5'd2, 16'd0 }      //lw
mem_instrucao[283] = { 6'd2, 5'd26, 5'd3, 16'd10 }    //addi
mem_instrucao[284] = { 6'd20, 5'd2, 5'd3, 5'd4, 11'd0 }     //cmp
mem_instrucao[285] = { 6'd19, 5'd4, 5'd26, 5'd4, 11'd0 }      //eq
mem_instrucao[286] = { 6'd25, 5'd4, 5'd26, 16'd18 }   //beq label8
mem_instrucao[287] = { 6'd4, 5'd25, 5'd5, 16'd0 }    //subi
mem_instrucao[288] = { 6'd8, 5'd5, 5'd5, 16'd0 }      //lw
mem_instrucao[289] = { 6'd1, 5'd27, 5'd5, 5'd6, 11'd0 }     //add
mem_instrucao[290] = { 6'd8, 5'd6, 5'd6, 16'd0 }      //lw
mem_instrucao[291] = { 6'd2, 5'd6, 5'd17, 16'd0 }    //addi
mem_instrucao[292] = { 6'd29, 5'd17, 5'd26, 16'd0 }     //out
mem_instrucao[293] = { 6'd4, 5'd25, 5'd8, 16'd0 }    //subi
mem_instrucao[294] = { 6'd8, 5'd8, 5'd8, 16'd0 }      //lw
mem_instrucao[295] = { 6'd4, 5'd25, 5'd9, 16'd0 }    //subi
mem_instrucao[296] = { 6'd8, 5'd9, 5'd9, 16'd0 }      //lw
mem_instrucao[297] = { 6'd2, 5'd26, 5'd10, 16'd1 }    //addi
mem_instrucao[298] = { 6'd1, 5'd9, 5'd10, 5'd11, 11'd0 }     //add
mem_instrucao[299] = { 6'd1, 5'd11, 5'd26, 5'd8, 11'd0 }     //add
mem_instrucao[300] = { 6'd4, 5'd25, 5'd25, 16'd0 }    //subi
mem_instrucao[301] = { 6'd9, 5'd8, 5'd25, 16'd0 }      //sw
mem_instrucao[302] = { 6'd2, 5'd25, 5'd25, 16'd0 }    //addi
mem_instrucao[303] = { 6'd23, 26'd280 }          //j label7
mem_instrucao[304] = { 32'd0 }  //label8
mem_instrucao[305] = { 6'd4, 5'd25, 5'd25, 16'd1 }    //subi
mem_instrucao[306] = { 6'd24, 26'd0 }      //jr
*/
```

6 Conclusão

Finalmente, é razoável constatar que o resultado obtido no projeto está de acordo com a proposta inicial. Mesmo com todos os problemas encontrados, de natureza teórica ou prática, foi possível desenvolver um compilador capaz de traduzir algoritmos em C- para linguagem de máquina.

As dificuldades encontradas, tanto teóricas como em aspectos práticos, contribuíram excessivamente no aumento do tempo de desenvolvimento do projeto. Uma vez superadas, é seguro afirmar que o desenvolvimento de outro projeto semelhante, envolvendo detalhes parecidos, e até de maior complexidade poderia ser realizado em um intervalo de tempo menor. Entre os empecilhos teóricos, os que apresentaram mais dificuldade foram repartir o processo de compilação em etapas menores, e definir corretamente, envolvendo todos os aspectos técnicos e de implementação, como cada etapa se comportaria e qual seria sua relação com as demais.

Entre os problemas técnicos encontrados, o modo de implementação, envolvendo a definição das estruturas realizadas e como elas se comportariam, foi o que exigiu mais tempo para ser solucionado. Tendo em vista todos os problema solucionados, é seguro afirmar que o projeto permitiu ao discente uma ampliação extremamente benéfica no conhecimento sobre compiladores e suas ferramentes, técnicas e aplicações.

Como próximos passos para o projeto do curso de Engenharia da computação, é necessário uma reestruturação em alguns trechos de código, de modo a otimizar o processo de compilação, e realizar uma integração apropriada com o processador desenvolvido, verificando e corrigindo possíveis bugs e problemas. Futuramente, também é possível que surjam outros requisitos e necessidades, para a utilização do projeto desenvolvido nas próximas disciplinas de Laboratórios de Sistemas Computacionais.

Referências

1 TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e Aplicações*. 11th edition. ed. São Paulo: Pearson Education, 2011. Citado na página 9.

2 V.AHO MONICA S. LAM, R. S. A.; ULLMAN, J. D. *Compiladores: Princípios, técnicas e ferramentas*. 2th edition. ed. São Paulo: Pearson Education, 2008. Citado 2 vezes nas páginas 9 e 17.

3 LOUDEN, K. C. *Compiladores - Princípios e práticas.* 1th edition. ed. São Paulo: Pioneira Thomson Learning, 2004. Citado 3 vezes nas páginas 10, 18 e 26.